Appunti di Linguaggi e Tecnologie per il Web

Federico Falcone

May 2, 2026

Contents

1 Networking **4**
1.1 Topologie . . . 4
1.2 Standard . . . 4
1.2.1 Standard ISO-OSI . . . 5
1.3 Protocollo . . . 5

2 HTML **7**
2.1 Linguaggio di markup . . . 7
2.2 HyperText Markup Language . . . 7
2.3 Browser . . . 7
2.4 Struttura della pagina HTML . . . 8
2.4.1 Tag . . . 8
2.5 Struttura della pagina . . . 8
2.5.1 Testa . . . 8
2.5.2 Corpo . . . 8
2.6 Identazione . . . 8
2.7 Annidamento . . . 8
2.8 Commenti . . . 9
2.9 Maioscolo o minuscolo . . . 9
2.10 Testa (`<head>`) . . . 9
2.10.1 Titolo . . . 9
2.10.2 Metadati . . . 9
2.10.3 Meta Tag - Keywords . . . 9
2.10.4 Attributo HTTP-EQUIV . . . 9
2.11 Corpo (¡body¿) . . . 10
2.12 Tag interni al body . . . 10
2.12.1 Elementi inline vs block . . . 10
2.12.2 Testo - Tag predefiniti . . . 10
2.12.3 Struttura della pagina con tag `<div>` . . . 11
2.12.4 Testo - Stili . . . 12
2.12.5 Testo - Font . . . 12
2.12.6 Elenchi . . . 13
2.13 Ipertestualità (Link) . . . 14
2.13.1 Link . . . 14
2.13.2 Destinazioni . . . 14
2.13.3 Caratteristiche dei link . . . 14
2.13.4 Manipolazione dei link . . . 14
2.13.5 Percorsi dei link . . . 14
2.13.6 Link interni o ancore . . . 15
2.13.7 Colorare i link . . . 15
2.13.8 Immagini . . . 15
2.14 Tabelle (table) . . . 16
2.14.1 Bordi . . . 16
2.14.2 Celle collassate in una riga/colonna . . . 16
2.15 Form . . . 17
2.15.1 Input . . . 17
2.15.2 Action attribute . . . 18
2.15.3 Method attribute . . . 18

2.15.4 Name attribute . . . 18
2.15.5 Fieldset . . . 18
2.15.6 Altri attributi . . . 19
2.16 XHTML . . . 20

3 Bootstrap **20**
3.1 Come ottenere Bootstrap? . . . 20
3.1.1 CDN di Bootstrap . . . 20
3.2 Creare una pagina Web con Bootstrap . . . 21
3.3 Griglie Bootstrap . . . 22
3.4 Nuovo Flexbox grid system . . . 23
3.5 Classi di griglia . . . 24
3.5.1 Struttura di base di una griglia Bootstrap . . . 24
3.6 Tabelle Bootstrap . . . 24
3.7 Immagini Bootstrap . . . 24
3.8 Bottoni Bootstrap . . . 25
3.8.1 Elementi del pulsante Bootstrap . . . 25
3.8.2 Dimensioni dei pulsanti . . . 25
3.8.3 Pulsanti a livello di blocco . . . 25
3.8.4 Pulsanti attivi/disabilitati . . . 25
3.9 Personalizzazione . . . 25
3.10 Cards . . . 25
3.11 Nuovi helpers . . . 26
3.12 Introduzione a SASS . . . 26
3.12.1 Come funziona . . . 26
3.12.2 Caratteristiche di Sass . . . 27
3.12.3 Nesting . . . 27
3.12.4 Variabili . . . 28
3.13 Appunti utili . . . 28

4 JavaScript **29**
4.1 Introduzione . . . 29
4.1.1 Applicazioni Web . . . 29
4.1.2 Applicazioni web: linguaggi . . . 29
4.1.3 Applicazioni web: linguaggi di programmazione e scripting . . . 29
4.1.4 Applicazioni Web . . . 30
4.1.5 Architettura server-side . . . 30
4.1.6 Architettura client-side . . . 31
4.1.7 AJAX . . . 31
4.2 JavaScript . . . 32
4.3 Tipi . . . 32
4.4 Variabili e istruzioni . . . 32
4.5 Variabili . . . 32
4.6 Tipo dinamico . . . 32
4.7 Istruzioni . . . 32
4.8 Costanti e commenti . . . 32
4.9 Operatori . . . 33
4.10 Specifica dello script . . . 33
4.11 Costrutto If-Else . . . 33
4.12 Liste . . . 33
4.13 Array . . . 33
4.14 Cicli . . . 34
4.14.1 Ciclo for . . . 34
4.14.2 Ciclo while . . . 34
4.15 Document Object Model (DOM) . . . 34
4.15.1 HTML DOM . . . 34
4.16 Oggetti (DOM) . . . 35
4.16.1 Oggetti (DOM): Liste di oggetti . . . 35
4.16.2 Oggetti (DOM): Window . . . 35
4.17 Window: componenti principale (estesi) . . . 36
4.18 WIndow . . . 36

4.19 Document . . . 36
4.20 Oggetti: proprietà e funzioni . . . 36
4.21 Accesso agli oggetti nella pagina . . . 37
4.22 Eventi . . . 37
4.23 Gestione eventi . . . 37
4.23.1 Eventi (form) . . . 37
4.23.2 Eventi (onLoad) . . . 39
4.23.3 Eventi (onMouseOver) . . . 39
4.24 Gestione finestre (open) . . . 39
4.25 Funzioni come link . . . 40
4.26 Accesso agli oggetti di una pagina tramite nodi . . . 40
4.26.1 Relazione tra nodi . . . 40
4.26.2 Aggiunta elementi . . . 41
4.26.3 Rimozione degli elementi . . . 41
4.26.4 Sostituzione eleemtni . . . 41
4.26.5 Collezione di nodi . . . 41
4.26.6 Metodo getElementsByClassName . . . 41
4.27 Funzioni . . . 42
4.27.1 Introduzione . . . 42
4.27.2 Definizioen di funzioni . . . 42
4.27.3 Chiamata a funzione . . . 42
4.27.4 Parametri di funzione . . . 42
4.28 Variabili: tipi di dichiarazione . . . 42
4.28.1 Variabili: dichiarazione esplicita . . . 42
4.28.2 Variabili: environment di riferimento . . . 42
4.29 Funzioni e chiusure . . . 42
4.29.1 Funzioni come dati . . . 43
4.30 Oggetti . . . 43
4.30.1 Definizione . . . 43
4.30.2 Costruzione . . . 44
4.30.3 Accesso alle proprietà . . . 44
4.30.4 Aggiunta e rimozione di proprietà . . . 44
4.30.5 Metodi per (singoli) oggetti . . . 44
4.30.6 Metodi per una "classe" di oggetti . . . 44
4.30.7 Metodi: invocazione . . . 44
4.31 Oggetti Function . . . 44
4.31.1 Definizione implicita . . . 44
4.31.2 Definizione esplicita . . . 45
4.32 Funzioni come dati: Esempio . . . 45
4.32.1 Funzioni come dati: problema . . . 45
4.33 Oggetti Function: proprietà . . . 45
4.34 Oggetti Function: metodi . . . 45
4.35 Funzioni come dati - Conseguenze . . . 46
4.36 Oggetto globale . . . 46
4.37 [Costruttori di] Oggetti predefiniti . . . 46
4.38 Date . . . 46
4.38.1 Costruzione . . . 46
4.38.2 Metodi . . . 46
4.39 Oggetto globale . . . 47
4.40 API . . . 47

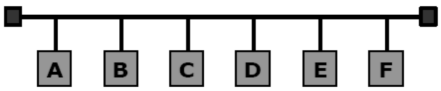
5 Node.js **47**
5.1 Programmi . . . 47
5.2 Gestire pacchetti con NPM . . . 47
5.3 Pacchetti locali vs globali . . . 47
5.3.1 Installare e disinstallare pacchetti . . . 48
5.4 Creare un semplice Server Web . . . 48
5.5 Leggere i file dal disco . . . 48
5.6 Creare un pacchetto . . . 48
5.6.1 Salvare le dipendenza di un pacchetto . . . 48
5.7 Usare NPM per eseguire un pacchetto . . . 49

1 Networking

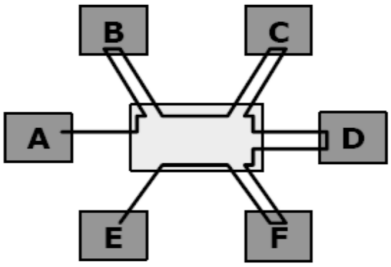
1.1 Topologie

Ogni tipologia di rete può essere fisicamente realizzata sfruttando topologie differenti:

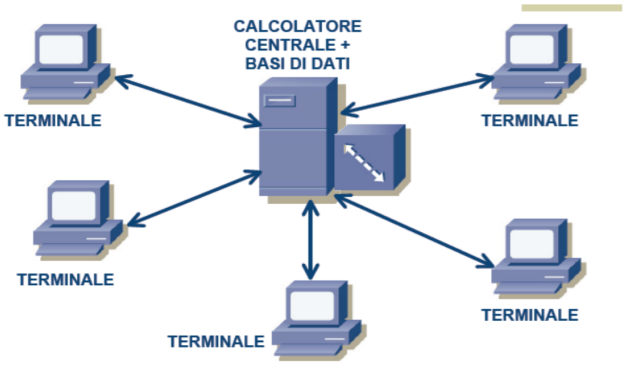
- topologia logica a Bus, cablata a stella

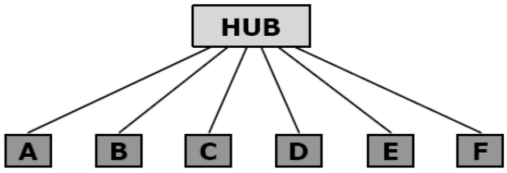


- topologia a stella



- centralizzata





1.2 Standard

La comunicazione tra nodi differenti e, possibilmente, basati su piattaforme hardware e/o software eterogenee necessita di standard. Storicamente si parla di standard di un produttore: scambio di informazioni possibile solo per sistemi della stessa marca (sistemi chiudi). Sono arrivati poi degli standard definiti da enti internazionali (livello di transizione dati).
Il vantaggio è che assicurano un vasto mercato per i dispositivi e il software. Inoltre consentono a prodotti provenienti da fornitori differenti di comunicare.
Di contro però è una tecnologia congelata e poi posono esserci più standard per la stessa funzione.

1.2.1 Standard ISO-OSI

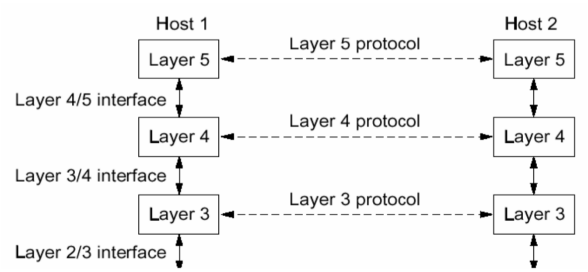
Lo standard ISO ha adottato un modello a strati. Il sistema di comunicazione viene diviso a strati ciascuno dei quali esegue una funzione predefinita. Strati ISO/OSI possono essere separati in due categorie

- funzioni dipendenti dalla rete (**Media layers**);
- funzioni orientate all'applicazione (**Host layers**).

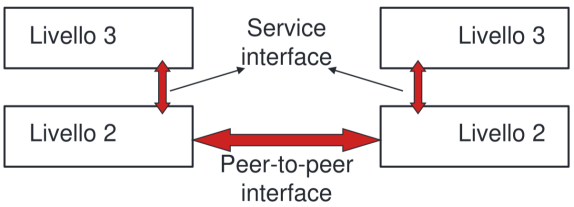
La funzione di ogni strato è codificata in una serie di regole e convenzioni usate per comunicare con lo strato remoto corrispondente (**protocollo**). Ogni strato fornisce dei servizi allo strato immediatamente superiore e utilizza inoltre dei servizi forniti dallo strato immediatamente inferiore.

1.3 Protocollo

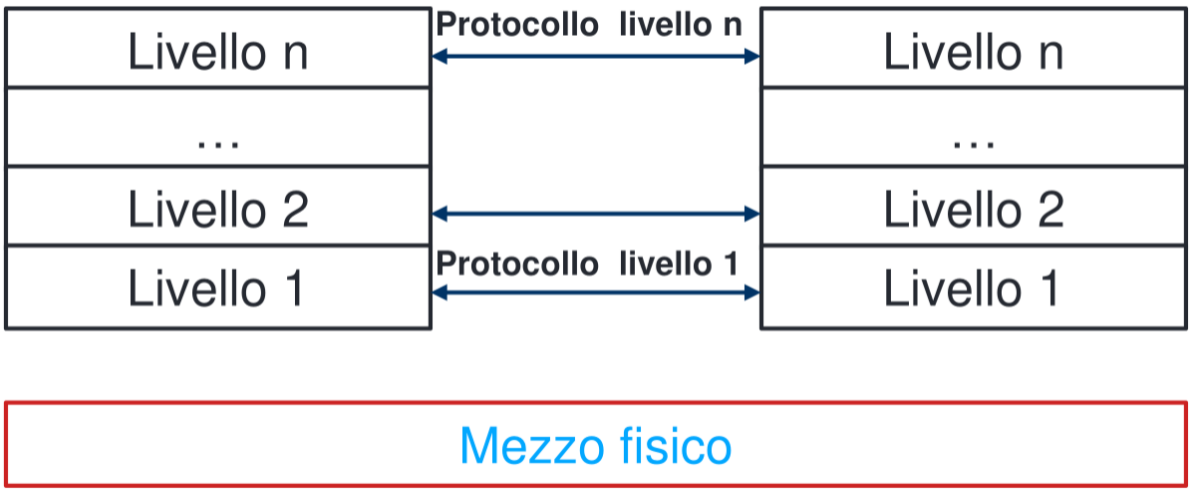
La comunicazione tra entità richiede cooperazione, ossia collaborazione per il conseguimento di uno scopo comune. Le comunicazioni sono regolate tramite protocolli.
Protocollo: insieme delle regole e convenzioni seguite da entità, dislocate su nodi distinti , che intendono comunicare per svolgere un compito comune. Tali regole hanno l'obiettivo di assicurare la cooperazione efficiente ed affidabile per la comunicazione tra nodi.
Sintassi: insieme e struttura dei comandi e delle risposte, formato dei messaggi.
Semantica: significato dei comandi, delle azioni, delle risposte da effettuare al momento della trasmissione e ricezione dei messaggi.
Temporizzazione: specifica delle possibilo sequenze temporali di emissione dei comandi e dei messaggi, nonchè eventuali risposte.
Ciascun protoccolo ha un'interfaccia "interna" verso il livello superiore o inferiore, e un'interfaccia "esterna" verso il livello equivalente di un altro modo.



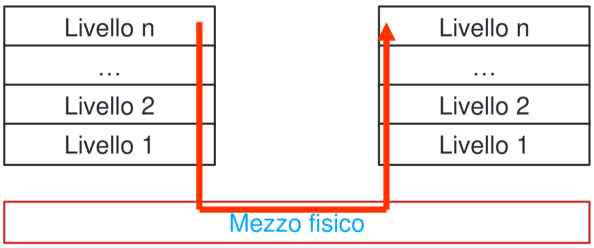
- **Service interface**("interna"):operazioni e servizi offerti al protocollo superiore.
- **Peer-to-peer interface**("esterna"): messaggi scambiati con un livello equivalente (peer) sull'altro nodo.



- Comunicazione logica tra *peer entity* (entità dello stesso livello).



- Comunicazione fisica (indiretta): la comunicazione tra *peer entity* è diretta solo a livello hardware.

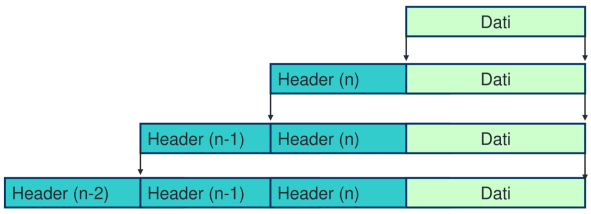


Un messaggio si compone di:

- **Header**: Protocol Control Information (PCI);
- **Dati**: Service Data Unit (SDU).

L'insieme delle due è chiamata **PDU** (Protocol Data Unit).

Incapsulamento del messaggio



La comunicazione avviene **logicamente** tra *peer*, ma in realtà attraversa tutti i livelli sottostanti, mediante **incapsulemnto del messaggio** a ciasun livello. Il sistema di comunicazione richiede un insieme di protocolli tra loro cooperanti (detti **protocol suite** o **protocol stack**).

2 HTML

2.1 Linguaggio di markup

Linguaggio di markup modellato per rendere esplicita una particolare interpretazione di un testo. Con il markup per sistemi informatici, specifichiamo le modalità esatte di utilizzo del testo nel sistema stesso. I linguaggi di markup sono i linguaggi più opportuni per strutturare e marcare i documenti in maniera indipendente dall'applicazione, favorendo la riusabilità, la flessibilità e la apertura ad applicazioni complesse.

2.2 HyperText Markup Language

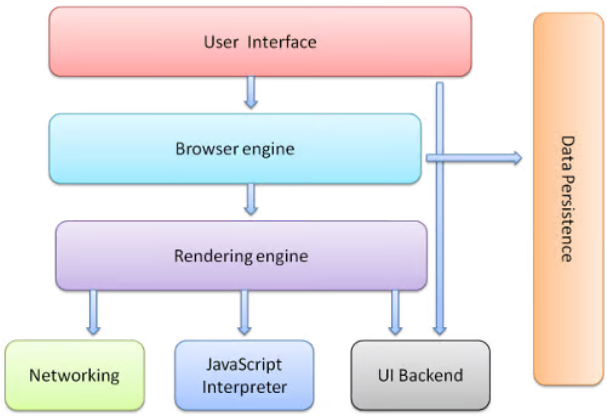
HTML è un linguaggio per la creazione di pagine Web.

- Standard per la rappresentazione.
- Descrive la struttura della pagina.
- Non è un linguaggio di programmazione, ma di markup.
- Descrive dati e regole su come mostrarli.
- Poche e semplici parole sintattiche.

2.3 Browser

È un programma per la navigazione Web. Richiede risorse attraverso il web (all'indirizzo richiesto) e la mostra nella finestra. Legge e inerpreta i documenti HTML, CSS, JavaScript, immagini sulla base di specifiche e standard.
Un Browser è formato da:

- **Layout Engine**: riceve input dal browser (URL bar, search box, mouse clicks and key presses) e li passa al rendering engine.
- **Rendering Engine**: riceve il codice HTML e lo interpreta visualizzandolo a video.
- **User interface**: controlli del browser, ad esempio, bottoni per andare avanti o indietro.
- **JavaScript engine**: engine che si prende cura di parsare ed eseguire codice JavaScript per poi ritornare il risultato dell'esecuzione.
- **Network Layer**: gestisce funzioni di rete, ad esempio, criptazione, richieste http, e tutte le configurazioni di rete.
- **Storage**: porzione di memoria dove il browser memorizza cached file, cookie, oggetti e dati create con JavaScript.
- **Operating System Interface**: componente che interagisce con il sistema operativo per disegnare diversi elementi e per gestire la finestra.



Il browser contiene anche tool per gli sviluppatori:

- Debug pagina.
- Analisi stili e sorgenti dei file.
- Visualizzazione HTTP request e response.
- Accesso a cookie e storage.
- Etc..

2.4 Struttura della pagina HTML

2.4.1 Tag

I tag sono marcatori che identificano porzioni di testo. Permettono la personalizzazione della pagina, e di identificare e processare le porzioni di testo contrassegnate da una certa marcatura.
Nome tag = nome funzione.
Il tag ha la seguente struttura

```
<tag attributi>contenuto</tag>
```

2.5 Struttura della pagina

Una pagina è formata da:

- **Riga di intestazione**: `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//IT">`. Indichiamo le specifiche W3C adottate. Esso dice al browser quale linguaggio è stato usato per il rendering.
- Il tag `<html>`: indica l'inizio di una pagina HTML. Tutto ciò compreso all'interno del tag `html` è il codice HTML.

```
<html>
    ...
</html>
```

- Documento HTML diviso in due:
  - Testa
  - Corpo

2.5.1 Testa

La testa contiene informazioni non immediate. Descrive come il documento deve essere letto e interpretato. Contiene meta-tag, script, stili.
È racchiuso nel tag `<head>`.

2.5.2 Corpo

Il corpo contiene il documento vero e proprio. Contiene tutti i tag per la realizzazione del sito Web.
È raachiuso nel tag `<body>`.

2.6 Identazione

Una buona norma è utilizzare caratteri di tabulazione. Aumenta la leggibilità, diminuendo così i tempi di modifica.

2.7 Annidamento

Una caratteristica di HTML sono i tag annidati.

2.8 Commenti

I commenti aumentano la leggibilità del codice. Una buona norma è commentare **solo** le parti significative. Permettono di orientarsi meglio all'interno del codice.

```
<!-- commento -->
```

2.9 Maioscolo o minuscolo

HTML è case unsensitive, mentre XHTML lo è. È consigliabile usare il carattere minuscolo.
Inoltre non è sensibile anche agli spazi e alle linee vuote.

2.10 Testa (`<head>`)

La testa contiene le informazioni/metadati relativi al documento. I browser di solito non presentano elementi che compaiono nella testa.

2.10.1 Titolo

L'elemento `<title>` permette di indicare il titolo della pagina.

2.10.2 Metadati

L'elemento `<meta>` definisce le informazioni relative alla pagina piuttosto che contenuto vero e proprio. Contiene 3 attributi principali:

- **Name**: contiene il nome della proprietà.
- **Content**: contiene il valore della proprietà.
- **http-equiv**: specifica le caratteristiche dell'HTTP responde header.

```
<META NAME="author" CONTENT="Federico">
```

2.10.3 Meta Tag - Keywords

I meta tag indicano alcune informazioni sul contenuto del sito. Sono separate da virgole, punto e virgola e spazio. Esse inoltre sono chiavi di ricerca.
Il motore di ricerca ricava le chiavi di indicizzazione. Bisogna fare attenzione nella scrittura delle keywords:

- evitare termini generici e particolari;
- riportare termini italiani ed inglese;
- singolare e plurale.

Evitare di ripetere le stesse parole e usare Keyword astute.

```
<meta name="keywords" content="calcio soccer tennis basket pallacanestro
    pallavolo volley inter milan juventus roma lazio...">
```

2.10.4 Attributo HTTP-EQUIV

Sono informazioni sulla comunicazione server-browser. Può essere utilizzato al posto dell'attributo name. L'attributo `http-equiv` fornisce un header HTTP per informazioni/valore dell'attributo content:

- Può essere usato per simulare un HTTP response header.
- Gli HTTP server usano questo attributo per ottenere informazioni circa l'HTTP response header.

Alcuni valori sono: `content-type, default-style, refresh`
Un esempio:

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
```

2.11 Corpo (¡body¿)

Il corpo di un documento contiene il contenuto vero e proprio. Il browser presenta il contenuto in diversi modi:

- Browser visuali: è una tela su cui disegnare testi, immagini, colori, ...
- Agenti audio: il contenuto stesso è parlato.

Esso contiene tutte le informazioni per la visualizzazione della pagina. Siccome gli style sheet sono il modo suggerito per specificare come verrà presentato un documento gli attributi del `body` sono stati deprecati.

2.12 Tag interni al body

2.12.1 Elementi inline vs block

Alcuni elementi HTML nel body sono "block-level", altri "inline". Gli elementi block-level possono contenere elementi inline e altri elementi block-level, mentre gli elementi inline contengono solo dati e altri elementi inline.

2.12.2 Testo - Tag predefiniti

I tag contenitori di testo sono block-level (ad eccezione di `<span>`).

- `<h1>,<h2>, ...` - heading;
- `<p>` - paragrafo;
- `<span>` - elemento inline per raggruppare porzioni di testo (ad esempio per colorarlo in maniera diversa rispetto al resto);
- `<div>` - è un contenitore degli elementi HTML.

Titoli - Heading
Sono previste sei grandezze predefinite per i titoli. Si va da `<h1>` che è la grandezza maggiore fino ad `<h6>` che è la grandezza minore.

```
<h1>Titolo</h1>
```

Paragrafo Il paragrafo è l'unità base di suddivisione del testo.

```
<p>Paragrafo</p>
```

`<p>` lascia una riga vuota prima e dopo il testo.

Blocco testo
`<div>` è un blocco contenitore. Esso va a capo ma non lascia righe di spazio.

```
<div>Blocco 1</div>
```

Non impone rappresentazioni di default.

Contenitore
È un contenitore generico e può essere annidato. È un elemento inline (continua sulla stessa riga del tag che lo contiene). Non va a capo.
Non impone rappresentazioni di default (a parte il fatto di essere un elemento inline).

Esempio

```
<h2>Titolo della pagina</h2>
<p>paragrafo 1</p>
<p>paragrafo 2</p>
<div>blocco 1</div>
<div>blocco 2
<span>contenitore 1</span>
<span>contenitore 2</span>
</div>
```

2.12.3 Struttura della pagina con tag `<div>`

`<div>` è uno tra i tag più utilizzati nella creazione di pagine Web, sopratutto nella creazione di layout. Esso fornisce un vero e proprio elemento strutturale della pagina. Suddivide gli spazi in zone per progettare il sito in modo semplice e dettagliato.
Molto utile quando usato insieme a fogli di stile.

Facciamo un esempio: immaginando di dover costruire un sito, pensiamo a come esso debba essere strutturato, tipicamente abbiamo:

- un contenitore (container);
- una parte alta (header);
- un corpo centrale (middle);
- un menù (navigation);
- una parte bassa (footer);
- spesso anche un pannello laterale (sidebar).

```
<div id="container">
<div id="header">
<div id="navigation"></div><!--#navigation-->
</div><!--#header-->
<div id="main"></div><!--#main-->
<div id="sidebar"></div><!--#sidebar-->
<div id="footer"></div><!--#footer-->
</div><!--#container-->
```

2.12.4 Testo - Stili

Sono possibili due tipi di stili:

- Fisici: definiscono lo stile grafico.
- Logici: forniscono informazioni sul ruolo svolto dal testo. Opzionalmente impongono uno stile grafico.

Stili fisici

- `<b>` bold: formatta il testo in **grassetto**;
- `<i>` italic: formatta il testo in *corsivo*;
- `<u>` underline: sottolinea il testo;
- `<strike>`: testo ~~barrato~~ (usato per correzioni);
- `<sup>` apice: $E = mc^2$;
- `<sub>` pedice: H_2O.

Stili logici

- `<abbr>` abbreviazione: non comporta nessun cambiamento grafico;
- `<address>` indirizzo;
- `<cite>` citazioni: testo in corsivo;
- `<samp>` esempio: testo a spaziatura fissa.

2.12.5 Testo - Font

Il carattere predefinito è il `Times New Roman`. Esso però è poco leggibile. Meglio `Verdana`, `Arial` o `Helvetica`.

```
<font face="verdana, arial, helvetica, sans-serif">testo</font>
```

È una buona norma utilizzare caratteri sicuri (sicuramente visualizzabili), non indicare un solo carattere e solo come ultima spiaggia sarà usato il `Times New Roman`.
Per scegliere il colore del testo:

```
<font color="#0000FF">testo blu</font>
```

LA scelta del colore e del carattere può essere fatta nello stesso tag.

```
<font color="#0000FF" face="verdana">testo blu</font>
```

Font annidati

```
<font color="blue" face="verdana">
    testo blu
    <font color="red">
        testo rosso
    </font>
</font>
```

Dimensione del testo

Si utilizza l'attributo `size` del tag `font`. Ci sono 2 modi per impostare la dimensione:

- Valore tra 1 e 7.

```
    <font size="3">Dimensione 3</font>
```

- Valori relativi alla dimensione del font.

```
    <font size="+2">Dimensione +2</font>
```

  Utilizza la dimensione del font di base. Il tag `<basefont>` permette di cambiare il font di base.

2.12.6 Elenchi

Gli elenchi possono essere:

- **ordinati**;
- **non ordinati**;
- **di definizioni**.

Sintassi:

```
<elenco>
    <elemento> primo elemento
    <elemento> secondo elemento
</elenco>
```

La chiusura dell'elemento è opzionale. La sintassi degli elenchi di definizione è leggermente diversa.

Elenchi ordinati
Gli elementi sono numerati progressivamente.
Il tag dell'elenco ordinato è `<ol>` (ordered list), mentre quello dell'elemento è `<li>` (list item).

```
<div>Titolo</div>
<ol>
    <li>Primo elem prima tabella
    <li>Secondo elem prima tabella
    <ol>
        <li> Primo elem seconda tabella
        <li> Secondo elem seconda tabella
    </ol>
</ol>
```

Elenchi non ordinati
Gli elementi non sono numerati. Si ottiene quindi un elenco puntato. Il tag è `ul` (unordered list).

```
<ul>
    <li> Primo elemento
    <li> Secondo elemento
    <ul>
        <li> Primo elemento
        <li> Secondo elemento
    </ul>
</ul>
```

Elenco di definizioni
Per creare un elenco di definizione si utilizza il tag `dl` (definition list) e il termine da definire `<dt>` (definition term). Per la definizione del termine si utilizza il tag `<dd>` (definition description).

```
<p> Titolo
<dl>
    <dt>&ltp&gt
        <dd>apertura paragrafo
    <dt>&ltdiv&gt
        <dd>apertura blocco di testo
    <dt>&ltspan&gt
        <dd>apertura elemento inline
    Altri tag...
</dl>
<dl>
```

2.13 Ipertestualità (Link)

2.13.1 Link

I link sono formati da due componenti:

- il contenuto (testo o immagine) che nasconde il collegamento;
- la risorsa puntata.

```
<a href="indirizzo"> link </a>
```

La testa del link è *link*, mente la coda è *indirizzo*.
La coda può essere una pagina HTML, un'immagine, documenti, altri file.

2.13.2 Destinazioni

Le destinazioni dei link possono essere:

- Pagina HTML: aperta nel browser.
- Immagine (.gif, .jpeg): aperta nel browser.
- Altri documenti (.doc, .pdf):
  - visualizzata nel browser;
  - richiesta di salvataggio.
- Altri file (.zip, .exe): richiesta di salvataggio.

2.13.3 Caratteristiche dei link

I link possono avere diversi stati:

- link a riposo: blu;
- link visitato: violetto;
- link attivo: passaggio da una pagina all'altra. Utile nel passato.
- link al passaggio del mouse: solo con fogli di stile.

2.13.4 Manipolazione dei link

- `<body link="red">`: cambia il colore dei link;
- `<body vlink="green">`: cambia il colore dei link visited;
- `body alink="yellow">`: cambia il colore dei link attivi.

2.13.5 Percorsi dei link

I percorsi dei link possono essere:

- assoluti: viene indicato il percorso per esteso;
- relativi: fanno riferimento alla posizione del file a cui si riferisce il link a partire dalla posizione della pagina che stiamo sviluppando.

Percorsi assoluti

```
<a href="http://www.miosito.it/cartella/miofile.html"> qui</a>
```

Percorsi relativi

```
<a href="sottodir/sottolink.html">Clicca qui</a>
```

2.13.6 Link interni o ancore

Permette di creare un indice.

- Ancora

```
    <a name="primo">ancora</a>
```

- Riferimento all'ancora

```
    <a href="#primo"> vai all'ancora </a>
```

- Riferimento ad inizio pagina

```
    <a href='#'>torna su</a>
```

2.13.7 Colorare i link

Per colorare solo un link:

```
<a href="...">
    <font color="green">
        link verde
    </font>
</a>
```

2.13.8 Immagini

Il web non è solo ipertesto, ma bensì ipermedia.

```
<img src="pic.jpg" alt="description" width="128"height="128">
```

- Attributo `src`: sorgente del file (percorso di memorizzazione file).
- Attributo `alt`: descrizione mostrata al passaggio sull'immagine.
- Attributi `width` e `height`: dimensioni dell'immagine.

Immagine come link:

```
<a href="destinazione.html">
    <img src="pic.jpg">
</a>
```

2.14 Tabelle (table)

Le tabelle si definiscono tramite l'elemento `<table>`. Ogni riga è definita con il tag `<tr>` contenuto in `table`. Ogni cella è definita con il tag `<td>` contenuto in `<tr>`.
Una riga può essere anche divisa in table heading usando l'elemento `<th>`. Di solito sono centrari e in grassetto.
Elemento `<caption>` serve per definire la caption della tabella.

```
<table style="width:100%">
    <tr>
        <td>Jill</td>
        <td>Smith</td>
        <td>50</td>
    </tr>
    <tr>
        <td>Eve</td>
        <td>Jackson</td>
        <td>94</td>
    </tr>
    <tr>
        <td>John</td>
        <td>Doe</td>
        <td>80</td>
    </tr>
</table>
```

2.14.1 Bordi

Per modificare i bordi della tabella si utilizza l'attributo `border`. Se non specificato nessun bordo:

```
<table border="1" style="width:100%">
```

2.14.2 Celle collassate in una riga/colonna

Si utilizza l'attributo `rowsspan` per collassare un riga:

```
<th rowspan="2">Telephone</th>
<td rowspan="2">Telephone</td>
```

Si utilizza l'attributo `colspan` per collassare una colonna:

```
<th colspan="2">Telephone</th>
<td colspan="2">Telephone</td>
```

2.15 Form

I form sono utilizzati per collezionare gli input degli utenti. È basato sull'elemento `<form>`. Esso specifica diversi elementi.

2.15.1 Input

È l'elemento più importante. Sono presenti diverse varianti a seconda da come si indica l'attributo `type`:

- `text`: definisce input testo normale;
- `radio`: definisce un input radio button (per selezionare una tra diverse scelte);
- `submit`: definisce un bottone submit (per sottomettere il form).

Text input
L'input text definisce un campo di testo di una linea. `<input type="password">` è un tipo speciale di campo testo dove il valore inserito viene nascosto.

```
<form>
    First name:<br>
    <input type="text" name="firstname">
    <br>
    Last name:<br>
    <input type="text" name="lastname">
</form>
```

Text area
Il text area definisce un campo testo a linee multiple.

```
<textarea name="message" rows="10" cols="30">
    The cat was playing in the garden.
</textarea>
```

Radio button
Il radio button definisce un input che permette **una** scelta tra diverse opzioni.

```
<form>
    <input type="radio" name="sex" value="male" checked>Male
    <br>
    <input type="radio" name="sex" value="female">Female
</form>
```

Checkbox input
La checkbox input definisce una serie di checkbox che permettono la scelta **multipla** tra diverse opzioni.

```
<form>
    <input type="checkbox" name="vehicle1" value="Bike"> I have a bike
    <br>
    <input type="checkbox" name="vehicle2" value="Car"> I have a car
</form>
```

Drop-down list
La drop down-drown list definisce un menù a tendina. L'elemento `<option>` definisce le possibili opzioni.

```
<select name="cars">
    <option value="volvo">Volvo</option>
    <option value="saab">Saab</option>
    <option value="opel">Opel</option>
    <option value="audi">Audi</option>
</select>
```

Submit button
La submit button definisce un bottone per sottomettere la form ad un handler. L'handler è una pagina lato server che processa i dati del form. L'handler è definito con l'attributo `action`.

```
<form action="action_page.php">
    First name:<br>
    <input type="text" name="firstname" value="Mickey">
    <br>
    Last name:<br>
    <input type="text" name="lastname" value="Mouse">
    <br><br>
    <input type="submit" value="Submit">
</form>
```

Permette anche di eseguire semplici azioni:

```
<button type="button" onclick="alert('Hello World!')">Click Me!</button>
```

2.15.2 Action attribute

L'attributo `action` definisce le azioni che devono essere fatte quando il form è inviato. La sottomissione viene comunque effettuata con il bottone submit. La pagina viene inviata a una pagina web lato server. Ad esempio uno script lato server gestisce la form inviata. Se l'attributo action è omesso, si considera la pagina corrente di default.

2.15.3 Method attribute

L'attributo `method` specifica l'operazione HTTP (GET o POST) da usare per inviare la form.
GET, è l'opzione di default, viene usata per l'invio di form passive e senza informazioni sensibili. I dati inviati sono visibili nell'indirizzo. È ottima per piccola quantità di dati.
GET è quindi utilizzata per l'invio di form che aggiornano dati, mentre POST fornisce maggiore sicurezza perchè i dati non sono visibili nell'indirizzo.

2.15.4 Name attribute

Ogni campo input deve avere un nome.

```
<form action="action_page.php">
    First name:<br>
    <input type="text" name="firstname" value="Mickey">
    <br>
    Last name:<br>
    <input type="text" name="lastname" value="Mouse">
    <br><br>
    <input type="submit" value="Submit">
</form>
```

2.15.5 Fieldset

Per raggruppare campi di una form, si utilizza `<fieldset>`. Si può definire anche una caption per la form `<legend>`.

```
<form action="action_page.php">
    <fieldset>
        <legend>Personal information:</legend>
        First name:<br>
        <input type="text" name="firstname" value="Mickey">
        <br>
        Last name:<br>
        <input type="text" name="lastname" value="Mouse">
        <br><br>
        <input type="submit" value="Submit">
    </fieldset>
</form>
```

2.15.6 Altri attributi

- `value`: definisce il valore iniziale;
- `readonly`: specifica che il valore non può essere cambiato;
- `disabled`: specifica un campo disabilitato e non utilizzabile (non viene inviato);
- `size`: specifica la dimensione in caratteri del campo;
- `maxlength`: specifica la dimensione massima del contenuto del campo.

2.16 XHTML

XHTML è quasi identito ad HTML, ma è più stringente. È supportato dai principali browser.

Perchè utilizzare XHTML
Molte pagine Web contengono HTML errato. Il seguente codice viene visualizzato bene nella maggior parte dei casi anche se non segue le regole di HTML:

```
<html>
    <head>
        <title>This is bad HTML</title>
    </head>
    <body>
        <h1>Bad HTML</h1>
        <p>This is a paragraph</p>
    </body>
</html>
```

Diversi browser che eseguo su device diversi richiedono maggiori risorse o potenza coputazionale per interpretare un markup sbagliato.
XHTML combina HTML e XML.

3 Bootstrap

Web design reattivo
Il responsive web design riguarda la creazione di siti web che si adattano automaticamente per avere un bell'aspetto su tutti i dispositivi, dai piccoli telefoni ai desktop di grandi dimensioni.
Bootstrap è il framework HTML, CSS e JavaScript più popolare per lo sviluppo di siti Web responsive e mobile-first.

Bootstrap è un framework front-end gratuito per uno sviluppo web più rapido e semplice. Bootstrap include modelli di progettazione basati su HTML e CSS per tipografia, moduli, pulsanti, tabelle, navigazione, modali, caroselli di immagini e molti altri, oltre al plug-in JavaScript opzionali.
Bootstrap dà anche la possibilità di creare facilmente design reattivi.

3.1 Come ottenere Bootstrap?

Ci sono due modi per iniziare ad usare Bootstrap:

- scaricare Bootstrap da `getbootstrap.com` e seguire le istruzioni da lì;
- includere Bootstrap da un CDN (Content Delivery Network).

3.1.1 CDN di Bootstrap

Bisogna includere i seguenti CSS, Javascript e jQuery di Bootstrap da MaxCDN nella pagina web:

```
<!-- Ultimo CSS Bootstrap compilato e minimizzato -->
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css
    /bootstrap.min.css">
```

```
<!-- JavaScript Bootstrap piu recente compilato -->
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js
    "></script>
```

```
<!-- ultima libreria jQuery -->
<script src="https://code.jquery.com/jquery-latest.js"></script>
```

Vantaggio dell'utilizzo di Bootstrap CDN: molti utenti hanno già scaricato Bootstrap da MaxCDN visitando un altro sito. Di conseguenza, verrà caricato dalla cache quando visitano il sito, il che porta a tempi di caricamento più rapidi. Inoltre, la maggior parte dei CDN farà in modo che una volta che un utente richiede un file da esso, verrà servito dal server più vicino a loro, il che porta anche a tempi di caricamento più rapidi.

3.2 Creare una pagina Web con Bootstrap

Bisogna aggiungere il doctype HTML5 siccome Bootstrap utilizza elementi HTML e proprietà CSS di questo doctype.

```html
<!doctype html>
<html lang="en">
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Bootstrap demo</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH" crossorigin="anonymous">
    </head>
    <body>
        <h1>Hello, world!</h1>
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz" crossorigin="anonymous"></script>
    </body>
</html>
```

Bootstrap è un mobile-first:

- Bootstrap 3 è progettato per rispondere ai dispositivi mobili. Gli stili mobile-first fanno parte del frameword principale.
- Per garantire un rendering e uno zoom al tocco corretti, bisogna aggiongere il tag `<meta>` all'interno dell'elemento `<head>`.

```html
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

- La parte `width=device-width` imposta la larghezza della pagina in modo che segue la larghezza dello schermo del dispositivo (che varierà a seconda del dispositivo).
- La parte `initial-scale=1` imposta il livello di zoom iniziale quando la pgina viene caricata per la prima volta dal browser.

Contenitori
Bootstrap richiede anche un elemento contenitore per racchiudere i contenuti del sito. Ci sono due classi di contenitori tra cui scegliere:

- La classe `.container` fornisce un **contenitore a larghezza fissa reattivo**.
- La classe `.container-fluid` fornisce un **contenitore a larghezza intera**, che copre l'intera larghezza del viewport.

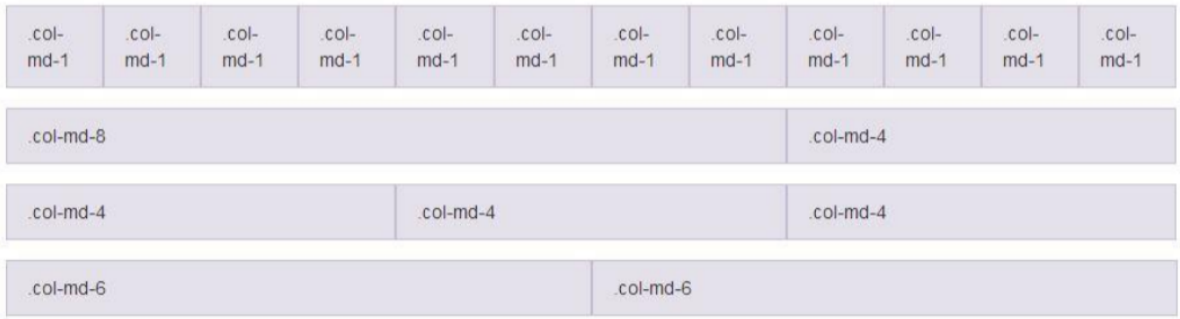
Nota: i contenitori non sono modificabili (non è possibile inserire un contenitore all'interno di un altro contenitore).

3.3 Griglie Bootstrap

Il sistema a griglia di Bootstrap consente fino a 12 colonne nella pagina. Se non si desira utilizzare tutte e 12 le colonne singolarmente, si possono raggruppare insieme per creare colonne più larghe.

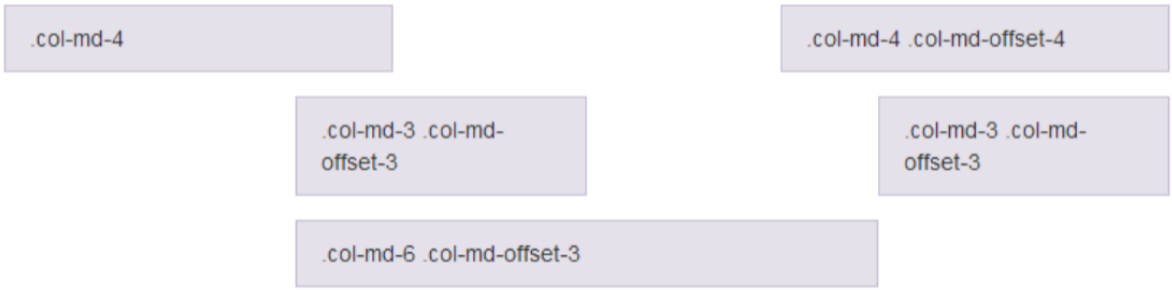
```
<div class="col-md-12">Si estende su 12 colonne</div>
<div class="col-md-6">Intervallo 6</div><div class="col-md-6">Intervallo 6</div>
<div class="col-md-4">Intervallo 4</div><div class="col-md-8">Intervallo 8</div>
<div class="col-md-4">Intervallo 4</div><div class="col-md-4">Intervallo 4</div>
    <div class="col-
md-4">Intervallo 4 </div>
```

Il sistema a griglia di Bootstrap è reattivo e le colonne si riorganizzeranno automaticamente a seconda delle dimensioni dello schermo.
È indivisuato quindi dalla classe `.col-(nome breakpont)-*`



È possibile innestare griglie nelle colonne creando delle `.row` nelle colonne, reiterando il meccanismo.

È possibile gestire i salti di colonna con gli offset `col-md-offset-*`.



Di default è possibe gestire 5 dimensioni del viewport:

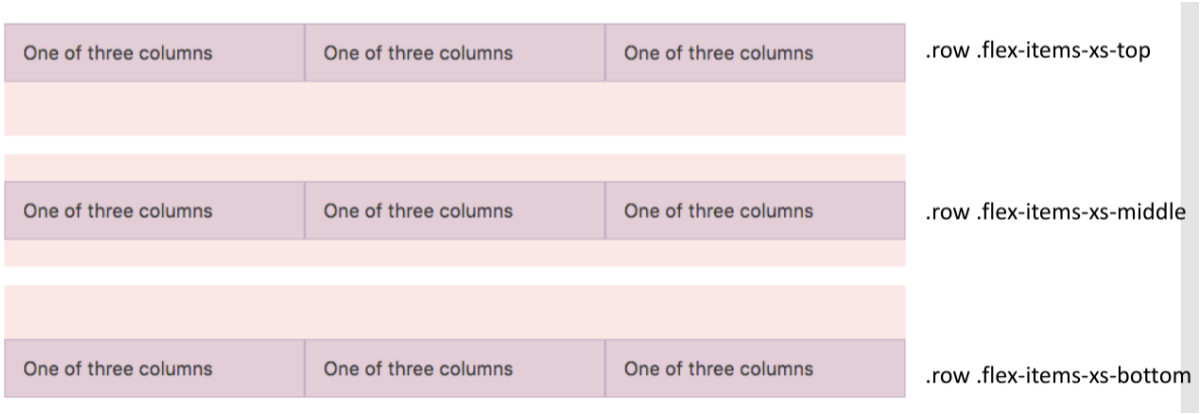
- Extra small (`.col-xs-*`), per gli smartphone.
- Small (`.col-sm-*`), per i tablet.
- Medium (`.col-md-*`), per i desktop.
- Large (`.col-lg-*`), per i desktop larghi.
- Extra large (`.col-xl-*`), per i desktop molto larghi.

3.4 Nuovo Flexbox grid system

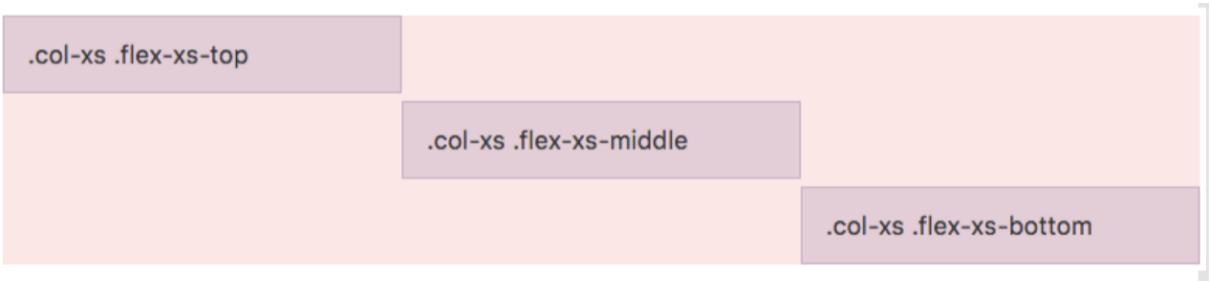
Con il nuovo Grid System non dobbiamo più indicare il numero nella classe della colonna, basterà usare solo la classe `.col-(nome breakpoint)` e si adatteranno in automatico allo spazio disponibile.

```
<div class="row">
    <div class="col-xs"> 1 of 3 </div>
    <div class="col-xs"> 2 of 3 </div>
    <div class="col-xs"> 3 of 3 </div>
</div>
```

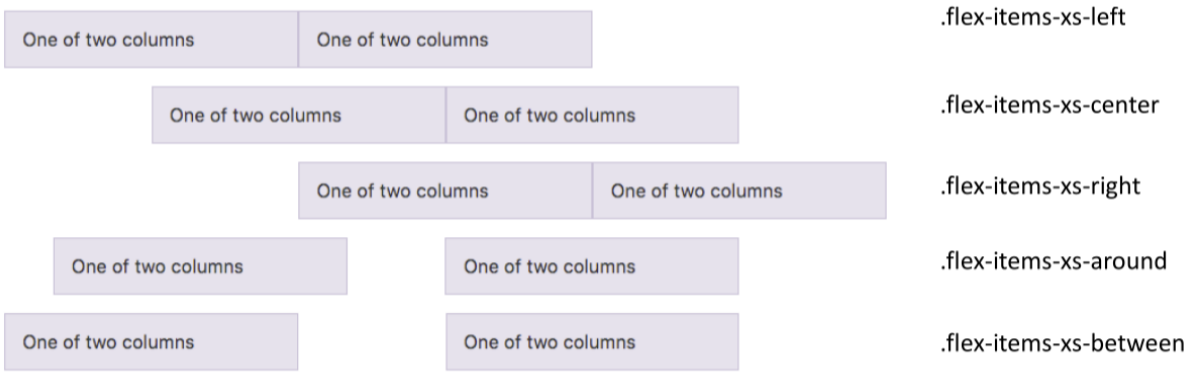
Possiamo allineare le righe in verticale:



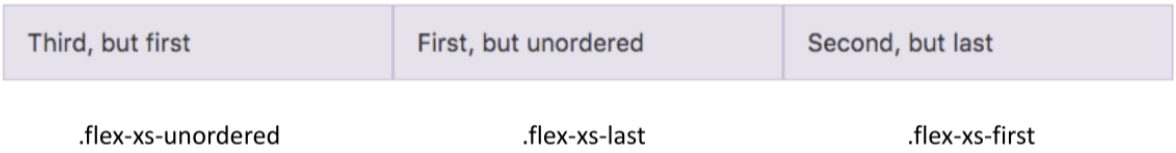
... e anche le colonne



Riordino orizzontale



Riordino della posizione



3.5 Classi di griglia

Il sistema di griglia di Bootstrap ha quattro classi:

- xs;
- sm;
- md;
- lg.

Le classi di cui sopra possono essere combinate per creare layout più dinamici e flessibili.

3.5.1 Struttura di base di una griglia Bootstrap

```
<div class="row">
    <div class="col-*-*"></div>
</div>
<div class="row">
    <div class="col-*-*"></ div>
    <div class="col-*-*"></div>
    <div class="col-*-*"></div>
</div>
<div class="row">
...
< /div>
```

Primo; creare una riga (`<div class="row"`). Quindi, aggiungere il numero desiderato di colonne (tag con classi `.col-*-*` appropriate). Nota che in numeri in `.col-*-*` dovrebbero sempre sommare fino a 12 per ogni riga.

3.6 Tabelle Bootstrap

Un tavolo Bootstrap di base ha una **leggera imbottitura e solo divisori orizzontali**. La classe `.table` aggiunge uno stile di base a una tabella:

- **righe a strisce**: la classe `.table-striped` aggiunge strisce zebrate a una tabella.
- **Tavolo bordato**: la classe `.table-bordered` aggiunge bordi su tutti i lati della tabella e delle celle.
- **Righe al passaggio del mouse**: la classe `.table-hover` abilita uno stato hover sulle righe della tabella.
- **Tabelle reattive**: la classe `.table-responsibe` crea una tabella reattiva. La tabella scorrerà quindi orizzontalmente su dispositivi di piccole dimensioni (768 px). Quando si visualizza su qualcosa di più largo di 768 px, non c'è differenza.

3.7 Immagini Bootstrap

- **Angoli arrotondati**: la classe `.img-rounded` aggiunge angoli arrotondati a un'immagine.
- **Cerchio**: la classe `.img-circle` modella l'immagine in un cerchio.
- **Miniatura**: la classe `.img-thumbnail` modella l'immagine in una miniatura.
- **Immagine reattive**: le immagini sono disponibili in tutte le dimensioni. Così fanno gli schermi. Le immagini reattive si adattano automaticamente alle dimensioni dello schermo. Crea immagine reattive aggiungendo una classe `.img-responsive` al tag `<img>`. L'immagine verrà quindi ridimensionata in modo corretto rispetto all'elemento genitore. La classe `.img-responsive` applica `display: block`; e larghezza massima: 100%: e altezza: `auto`; all'immagine.

3.8 Bottoni Bootstrap

Stili dei pulsanti: Bootstrap fornisce 7 stili di pulsanti con le seguenti classi:

- `.btn-predefinito`;
- `.btn-primary`;
- `.btn-success`;
- `.btn-info`;
- `.btn-warning`;
- `.btn-danger`;
- `.btn-link`.

3.8.1 Elementi del pulsante Bootstrap

Le classi di pulsanti possono essere utilizzaate sui seguenti elementi:

- `<a>`;
- `<pulsante>`;
- `<input>`.

3.8.2 Dimensioni dei pulsanti

Bootstrap fornisce quattro dimensioni di pulante con le seguenti classi:

- `.btn-lg`;
- `.btn-md` (dimensione predefinita);
- `.btn-sm`;
- `.btn-xs`.

3.8.3 Pulsanti a livello di blocco

Un pulsante a livello di blocco copre l'**intera larghezza** dell'elemento genitore. Per fare ciò si aggiunge la classe `.btn-block`.

3.8.4 Pulsanti attivi/disabilitati

Un pulsante può essere impsotato su uno stato attivo (appare premuto) o disabilitato (non cliccabile): la classe `.active` fa apparire premuto un pulsante e la classe `.disabled` rende un pulsante non cliccabile.

3.9 Personalizzazione

Il codice sorgente di Bootstrap può essere personalizzato:

- Attraverso un tool online (V3).
- Less (V3).
- Sass (V3 e V4).

3.10 Cards

Wells, thumbnails e panels ci lasciano in favore delle cards.

3.11 Nuovi helpers

Nuovi classi di supporto per gestire margin, padding e proprietà display

- `.pb-0` (padding-bottom: 0;);
- `-p-3` (padding: 3rem;);
- `mt-0` (margin-top: 0;);
- `.ml-1` (margin-left: 1rem;);
- `.mx-auto` (margin-left: auto; margin-right: auto;);

Migliorate le classi di supporto per allineare il testo in base al breakpoint:

- `text-xs-left` (-right, -center);
- `text-sm-left` (-right, -center);
- `text-md-left` (-right, -center);
- `text-lg-left` (-right, -center);
- `text-xl-left` (-right, -center);

Migliorate le classi di supporto per allineare gli elementi in base al breakpoint:

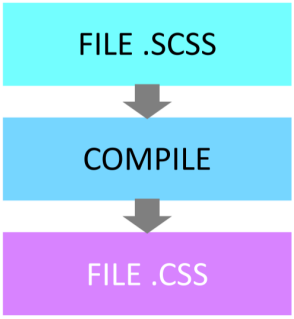
- .float-xs-left (-right);
- .float-sm-left (-right);
- .float-md-left (-right);
- .float-lg-left (-right);
- .float-xl-left (-right);

Migliorate le classi per nascondere o mostrare elementi in base al breakpoint: `.hidden-xs` diventa `.hidden-xs-up`.
Ad esempio: `.hidden-md-up` nasconderà tutti gli elementi dal breakpoint md in su.

3.12 Introduzione a SASS

SASS: è un **pre-processore CSS**, un meta-linguaggio che ci permette di usare una sintassi più simile a quella dei linguaggi di programmazione offrendo moltissimi vantaggi tra cui una **maggiore manutenibilità del codice**. SASS è nato per rimediare a disordine e ripetizioni, per introdurre una strututra formale dei CSS, migliorare la scrittura del codice e renderlo più manutenibile e ottimizzare la gestione delle media queries e andare incontro alle esigenze del mercato in crescita di dispositivi mobili.

3.12.1 Come funziona



3.12.2 Caratteristiche di Sass

- **Nesting**: possibilità di nidificare i selettori ottenendo un codice pulito.
- **Variabili**: come in altri linguaggi è un insieme di dati modificabili istuati in una porzione di memoria;
- **Importazione**: permette di importare altri file `scss` o `css` senza generare un grosso numero di richieste HTTP che rallenterebbero inevitabilmente il sito.
- **Funzioni**: in Sass esistono dei set di funzioni preconfezionate per effettuare diverse operazioni, è inoltre possibile sviluppare funzioni personalizzate;
- **Mixin**: permette di realizzare dei set riutilizzabili di regole CSS;
- **Logica condizionale**: @if, @else, @else if.
- **Cicli**: @for, @each, @while.
- **Compass, scount, ecc...**: framework opensource che offrono un'ampia gamma di mixin, funzioni e variabili preconfezionati.

3.12.3 Nesting

Con il **nesting** basta dichiarare una sola volta il nome dell'elemento radice per inestare le altre regole figlie all'interno.

```
#box{
    background-color: red;

    header{
        background-color: yellow;

        h1{ color: black; }
    }

    section h1{ color: blue; }
}
```

```
#box{
    background-color: red;
}

#box header{
    background-color: yellow;
}

#box header h1{
    color: black;
}

#box section h1{
    color: blue;
}
```

SASS fornisce il selettore speciale "**&**" che, utilizzato all'interno di un **nesting** farà riferimento all'elemento radice.

```
#box {

    &-main {
        width: 100%
    }
    &.small {
        width: 100%
    }
}
```

```
1  #box-main {
2      width: 100%;
3  }
4  #box.small {
5      width: 100%;
6  }
7
8
9
```

"&" può essere utilizzato per definire le pseudoclassi di un elemento.

```
a {
    color: #000;
    &:hover{
    color: #aaa;
    }
}
```

```
1  a {
2      color: #000;
3  }
4  a:hover {
5      color: #aaa;
6  }
7
```

3.12.4 Variabili

Variabili: come nei linguaggi di programmazione il concetto di variabile è pressoché identico, una variabile **SASS** è un contenitore virtuale di:

- stringhe di testo;
- valori numerici;
- numeri in virgola mobile;
- valori booleani.

Ad esempio possiamo creare dei set di colori:

```
$base-color: #abcdef;

button {
    background: $base-color;
}
a {
    color: $base-color;
}
header {
    border-bottom: 5px;
    border-color: $base-color;
}
```

```
button {
  background: #abcdef;
}

a {
  color: #abcdef;
}

header {
  border-bottom: 5px;
  border-color: #abcdef;
}
```

Oppure gestire facilmente le media queries.

3.13 Appunti utili

- Se voglio fare in modo che un testo sia colorato di un certo colore, utilizzare la classe `.text-white`.
- Per dare un spazio tra un elemento e l'altro, utilizzare le classi di **spacing**.
- Al posto di `<small>` si può usare la classe `.small`.
- Utilizzare `<span>` per raggruppare un testo e applicare uno stile specifico.
- Per cambiare il colore si può usare anche la classe `.text-*colore*`. Aggiungedoci -*valore* si aggiunge la trasparenza.
- Per modificare font testo, utilizzare `.fw-fold`, ad esempio.
- Per creare un testo con la sidebar, utilizzare `overflow-auto`.
- Per far sì che una immagine entri correttamente in un contenitore, utilizzare la proprietà CSS `object-fit: cover;`.

4 JavaScript

4.1 Introduzione

4.1.1 Applicazioni Web

Le risorse in rete identificata tramite URL è di 3 tipi:

- pagina **statica** (HTML) il cui contenuto è **fisso**, definito nel momento in cui la pagina è stata scritta;
- pagina **dinamica** il cui contenuto viene generato (selezionato, composto) al momento della richiesta (o della visualizzazione);
- **applicazione web**: software sviluppato e utilizzato attraverso attraverso tecnologie web;
  - linguaggi di mark-up, scripting e programmazione;
  - teconologie client-side e server-side;

4.1.2 Applicazioni web: linguaggi

- linguaggi di mark-up descrivono documenti strutturati (contenuto + struttura).
- linguaggi di programmazione descrivono programmi come sequenza di istruzioni;
- linguaggi di scripting, tipo particoalre di linguaggio di programmazione, per scrivere script.

4.1.3 Applicazioni web: linguaggi di programmazione e scripting

Un programma è un insieme di istruzioni, mentre i linguaggi di programmazione sono linguaggi per scrivere programmi. Un linguaggio è composto da:

- **Alfabeto**: insieme di simboli con cui si possono costruire i termini del linguaggio (lessico).
- **Sintassi**: definita da una grammatica che fornisce regole di composizione dei termini in frasi bene formate del linguaggio.
- **Semantica**: definisce il significato delle frasi ben formate del linguaggio.

Un **analizzazatore sintattico** (parser) analizza frasi e decide se sono frasi ben formate del linguaggi o no.
Lingiaggio di programmazione:

- lessico: "keywords" del linguaggio;
- frasi ben formate: istruzioni (programmi);
- semantica: esecuzione del programma.

Il programmatore scrive il programma sorgente utilizzando un linguaggio ad "alto livello". Occore però tradurre il programma sorgente in un programma composto da istruzioni in un linguaggio macchina (programma eseguibile). Le tecniche per effettuare questa traduzione sono:

- **Compilazione**: traduzione effettuata da strumenti chiamati compilatori.
- **Interpretatori**: traduzione effettuata da strumenti chiamati interpreti.
- **Approccio misto**.

La traduzione effettuata dal compilatore è specifica per una data macchina. Inoltre genera un file eseguibile (.exe). Il vantaggio è che è efficiente. Lo svantaggio è che è poco flessibile.
La traduzione effettuata dall'interprete interpreta un programma (traduzione simultanea). Ogni istruzione viene tradotta in un insieme di istruzioni nel linguaggio macchina della piattaforma specifica ed eseguita. C'è un interprete specifico per ogni piattaforma. Il vantaggio è che è flessibile, di contro però è poco efficiente.
Nell'approccio misto, il programma sorgente viene tradotto in formato intermedio (bytecode) indipendente dalla macchina (compilatore). Il programma in formato intermedio verrà poi interpretato dall'interprete. L'approccio misto somma i vantaggi dei due approcci. È efficiente e flessibile.

4.1.4 Applicazioni Web

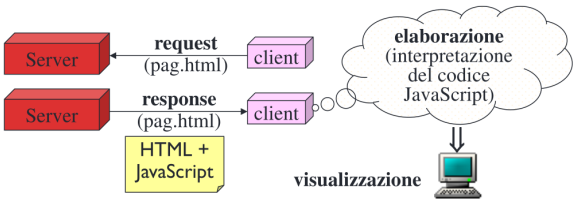
Pagine web "dinamiche": il contenuto viene generato al momento della richiesta/visualizzazione. Si basano su linguaggi di programmazione e scripting:

- tecnologie client-side (pagine "debolmente" dinamiche):
  - elaborazione lato client (browser);
  - scripting client-side (JavaScript), Java Applet, Adobe Flash.
- tecnologie server-side (pagine "fortemente" dinamiche):
  - elaborazione lato server (web server);
  - scripting server-side (PHP, Active Server Pages, Java Server Page), programmi (Java Servlet), Ruby, Python, Perl, (Javascript).

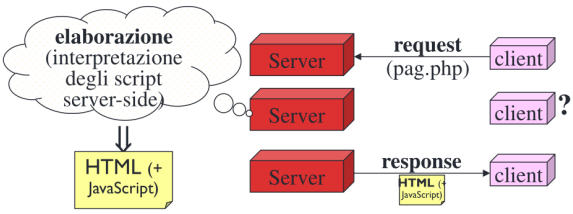
Le applicazioni web si sviluppano su 3 livelli logici:

- Presentazione - HTML, CSS.
- Intermedio - JavaServlet, JSP, PHP, ASP, JavaScript, Flash.
- Dati - RDBMS, XML, JSON.

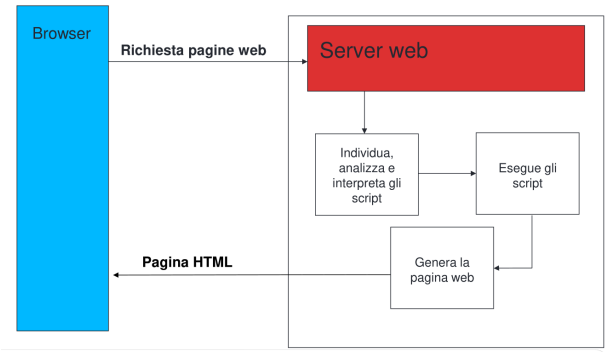
Client-side:



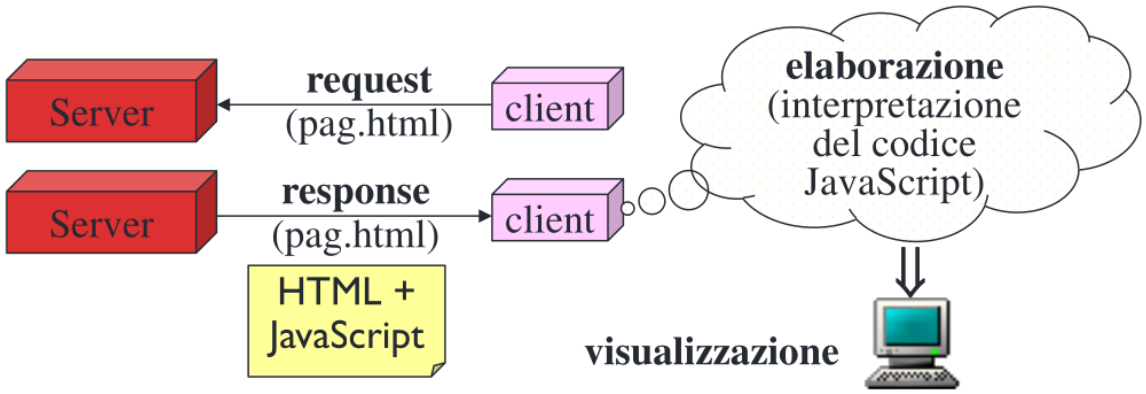
Server-side:



4.1.5 Architettura server-side



4.1.6 Architettura client-side

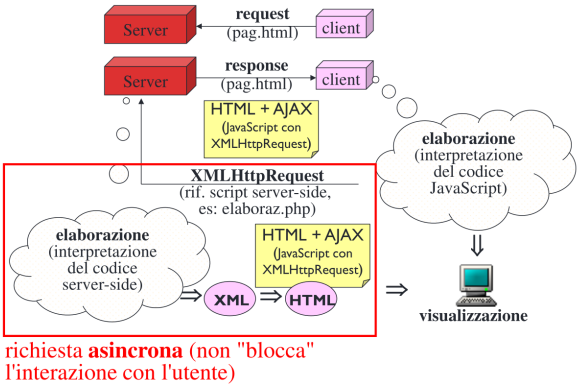


Per visualizzare una pagina Web "debolmente" dinamica NON HO bisogno di un server: posso aprire la pagina fornendo al browser il path sul file system locale.
Per visualizzare una pagine Web autenticamente dinamica H(che utilizza una tecnologia server-side) HO bisogno di un server: devo connettermi al server (e richiedere la pagina) tramite un URL.
Se chiedo al browser di visualizzare il codice sorgente della pagina:

- Nel caso di una pagina Web "debolmente" dinamica vedo l'HTML + il codice "dinamico" client-side.
- Nel caso di una pagina Web autenticamente dinamica vedo solo l'HTML: al posto del codice "dinamico" server-side, il server ha infatti sostituito il risultato dell'alaborazione (cioè codice HTML).

Esistono degli approcci ibridi, utilizzando sia tecnologie client-side che teconologie server-side. Un esempio è AJAX e JQuery.

4.1.7 AJAX



4.2 JavaScript

JavaScript è un linguaggio di scripting, tipicamente utilizzato client-side. Nonostante la somiglianza nel nome, è un linguaggio completamente distinto da Java. Come tutti i linguaggi di scripting, è interpretato: il sorgente non deve essere compilato per essere eseguito. L'interprete di JavaScript è generalmente contenuto all'interno del browser.
A differenza di HTML, JavaScript è case-sensitive.
Putroppo possono esserci incompatibilità e differenze tra i diversi browser: a volte si comportano in maniera diversa o non funzionano.
Si basa su due concetti principali:

- DOM (Document Object Model).
- Eventi (script event-driven).

4.3 Tipi

JavaScript è un linguaggio debolmente tipato. Il tipo delle variabili non viene dichiarato esplicitamente, ma definito implicitamente al primo assegnamento. Javascript converte automaticamente i tipi durante l'esecuzione (quando possibile).
Tipi principali in JavaScript:

- Number: interi e decimali (virgola mobile);
- Boolean: valori booleani;
- String: sequenze di caratteri.

4.4 Variabili e istruzioni

Nella dichiarazione delle variabili non è obbligatorio l'uso della keyword `var`, inoltre sono senza tipo: `var variabile`.
Nella dichiarazione delle variabili locali è obbligatorio l'uso della keyword `var` e sono senzaa tipo.
Le inizializzazioni sono facoltative (ma è buona norma farle all'atto della definizione e prima di utilizzare le variabili). Tutte le istruzioni JavaScript devono terminare con ;.

4.5 Variabili

Sono loosely typed: è possibile assegnare a una stessa variabile prima un valore stringa, poi un numero poi un altro ancora.
Due possibili scope:

- globale, per le variabili definite fuori da funzioni;
- locale, per le variabili definite esplicitamente dentro a funzioni (compresi i parametri ricevuti).

ATTENZIONE: un blocco NON delimita uno scope! Tutte le variabili definite fuori da funzioni, anche se dentro a blocchi innestati, sono globali.

4.6 Tipo dinamico

L'operatore `typeof` ritorna il tipo di una espressione. Il tipo è dinamico: rappresenta il tipo di quel momento temporale e corrispondente al valore attuale della variabile.

4.7 Istruzioni

Le istruzioni devono essere separate da un fine riga o da un punto e virgola.

4.8 Costanti e commenti

Le costanti numeriche sono sequenze di caratteri numerici non racchiuse da virgolette o apici.

4.9 Operatori

Gli operatori sono:

- aritmetici: +, -, *, /, ++, –;
- di confronto: ==, !=, ¿, ¡, ¿=, ¡=, === (fa anche il controllo del tipo);
- booleani: &&, ——, !;
- assegnamento: =;

4.10 Specifica dello script

Il codice di un programma JavaScript viene incluso in un file HTML per mezzo del tag `<SCRIPT>`. Sono possibili più programmi nella stessa pagina.
Le definizioni delle funzioni vengono generalmente incluse nella sezione `<HEAD>`. I richiami alle funzioni (predefinite nel linguaggio o definite dal programmatore) avvengono dove occorre, nel body della pagina HTML.

4.11 Costrutto If-Else

Il costrutto If-Else serve per esprimere un'azione condizionale.

```
if (condizione1){
    sequenza_di_azioni_1
}
else if (condizione2){
    sequenza_di_azioni_2
}
else {
    sequenza_di_azioni_n
}
```

4.12 Liste

Una lista è una sequenza ordinata di elementi. Le operazioni sono:

- calcolo lunghezza;
- stampa di tutti gli elementi;
- aggiunta di un elemento;
- cancellazione di un elemento.

4.13 Array

Un array è una lista rappresentata come array con indice a partire da 0. Le celle diipt non hanno il vincolo di omogeneità in tipo: ogni cella può contenere indistintamente numeri, stringe, oggetti, altri array, ...
Creazione e inizalizzazione di un array:

```
var lista=new Array();
lista[0]="caludio
```

Oppure lo si può inizializzare in fase di definizione:

```
lista=new Array("Claudio", "Erika", "Denise");
```

Inserimento di valori: i singoli elementi sono referenziaati con l'usuale notazione a parentesi quadre:

```
lista[i]="stringa";
```

La lunghezza di un array si ottiene:

```
var lunghezza=lista.length;
```

Per (sovra)scrivere l'ultimo elemento dell'array:

```
lista[lunghezza-1]="stringa";
```

Ogni scrittura sovrascrive l'elemento che era memorizzato in precedenza.

4.14 Cicli

Strutture di controllo per l'accesso sequenziale a un set di elementi. I cicli sono generalmente basati sul concetto di indice: l'indice scorre lungo la lista indicando, via via, posizioni successive.

4.14.1 Ciclo for

```
for(inizio;test;incremento){
    istruzioni
}
```

Le istruzioni eseguite a partire da inizio, finchè test è vero, avanzando ad ogni passo di quanto è indicato da incremento.

4.14.2 Ciclo while

```
while(condizione){
    istruzioni
}
```

Finchè la condizione è vera esegui le istruzioni.

4.15 Document Object Model (DOM)

Definisce uno standard per accedere ai documenti. È separato in 3 parti:

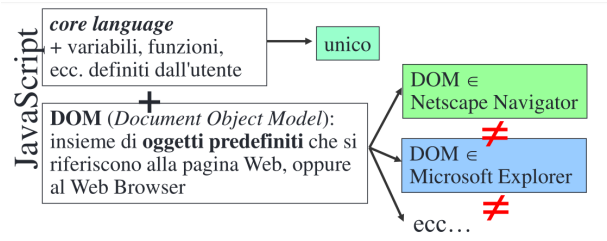
- core DOM: modello standard per tutti i tipi di documenti;
- XML DOM: modello standard per documenti XML;
- HTML DOM: modello standard per documenti HTML.

4.15.1 HTML DOM

È uno standard object model e programing interface per HTML. Esso definisce:

- Elementi HTML come oggetti.
- Proprietà degli elementi HTML.
- I metodi per accedere agli elementi HTML.
- Gli eventi per tutti gli elementi HTML.

In altre parole, è uno standard che definisce come ottenere, cambiare aggiungere o modificare elementi HTML.



JavaScript usa HTML DOM per modificare tutti gli elemetni di una pagina Web:

- Quando una pagina è caricata il browser crea il DOM della pagina.
- La pagina è rappresentata come un albero.
- DOM è definito dal browser.

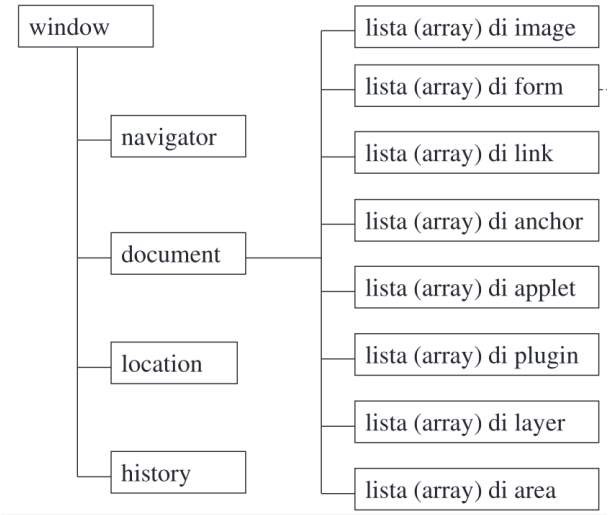
4.16 Oggetti (DOM)

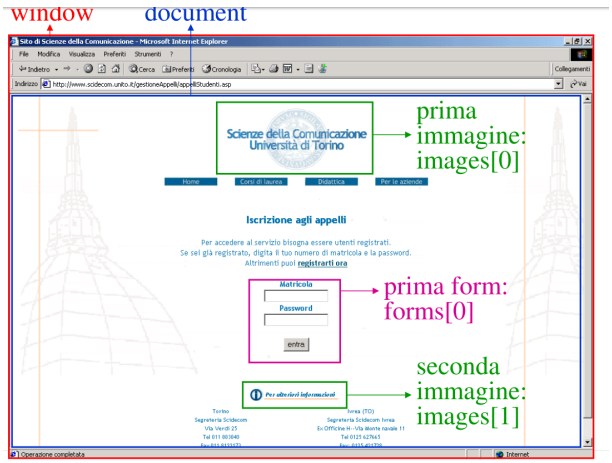
Tramite il modello DOM a oggetti, Javascript può:

- Cambiare tutti gli elementi e attributi HTML di una pagina.
- Cambiare tutti gli stili CSS.
- Rimuovere elementi e attributi HTML.
- Aggiungere elementi e attributi HTML.
- Reagire a eventi nella pagina.
- Creare nuovi eventi.

4.16.1 Oggetti (DOM): Liste di oggetti

Il DOM è organizzato seconod una gerarchia:





4.16.2 Oggetti (DOM): Window

Windows (this) = la finestra corrente del browser. Figli dell'oggetto window:

- navigator: il browser;
- document: il contenuto della finestra;
- location: informazioni sull'URL corrente;
- history: elendo degli URL visitati di recente.

4.17 Window: componenti principale (estesi)

- self;
- windows;
- parent;
- top;
- navigator;
- frames (array);
- location;
- history;
- document;

4.18 WIndow

Radice della gerarchia DOM: rapprenta la finestra del browser e fornisce diversi metodi.
Metodo Alert: `window.alert(messaggio)` propone una finestra di avviso con messaggio. La funzione alert è usabile anche in un link e non restituisce nessun valore.
Altri metodi:

- confirm, che fa apparire una finestra di conferma con il messaggio dato: restituisce `true` o `false`.
- prompt, che fa apparire una finestra di dialogo per immettere un valore: resituisce il valore string.

4.19 Document

Rappresenta il documento corrente: il contenuto della oagina web attuale. I componenti principali di document sono:

- forms (array): lista dei moduli (form) contenute nella pagina;
- elements (array di Buttons, Checkboc, ...)
- anchors (array);
- links (arrays);
- images (array): lista delle immagini contenute nella pagina;
- applets (array);
- ambeds (array).

Diversi metodi disponibili: `document.write` stampa un valore a video.

4.20 Oggetti: proprietà e funzioni

Un oggetto è una collezione di:

- proprietà (variabili): sono a loro volta oggetti;
- funzioni (metodi, operazioni).

L'invocazione di una funzione apparentemente senza alcun oggetto si riferisce all'oggetto window. Un riferimento all'oggetto document non preceduto da un riferimento all'oggetto window, è equivalente al caso in cui document è preceduto da window (implicito).

4.21 Accesso agli oggetti nella pagina

Lista dei moduli contenuti in una pagina: `window.cocument.forms[i]`.
Lista delle immagini contenute in una pagina: `window.document.images[i]`.
I campi di testo sono oggetti dotati di nome posti all'interno di un oggetto form pure esso dotato di nome.
Come tali sono referenziabili con la "dot notation": `documen.nomeform.nomeTextField`.
Il campo di testo è caratterizzato dalla proprietà values.
Esempio

```
<FORM name="myform">
<INPUT type="text" name="cognome" size=20>
<INPUT type="button" name="bottone" value="Show"
onClick="alert(document.myform.cognome.value)">
</FORM>
```

Attributo NAME:

```
<INPUT TYPE="text" NAME="codice_fiscale">
```

```
window.document.modulo.codice_fiscale.value=...
```

Metodo `getElementById(idname)`: ritorna l'elemento con uno specifico id. Ritorna `null` se non esiste l'elemento.

```
<INPUT TYPE="text" ID="codice_fiscale">
```

```
window.document.getElementById("codice_fiscale").value=...
```

4.22 Eventi

I programmi JavaScript sono tipicamente "guidati dagli eventi" (event-driven). Gli eventi sono scatenati da azioni dell'utene sulla pagina Web: ogni volta che l'utente scrive qualcosa in una casella, preme un pulsante, ridimensiona una finestra ecc... genera un "evento".
Un programma JavaScript deve contenere un gestore di eventi (event handler), che sia in grado di ricevere e interpretare le azioni dell'utente (eventi). Il DOM fornisce una serie di gestori di eventi predefiniti: l'accadere di un evento nella pagine Web mette automaticamente in azione il corrispondente gestore di eventi.

```
<a href="#" onClick = "window.print()" >
```

Attributo `onClick`:evento click innesca il gestore.
Un'istruzione JavaScript può essere inserita all'interno di un tag HTML. In questi casi, il gestore di evento invocato farà riferimento al tag in cui si trova l'istruzione.
Gli eventi intercettabili su un link: `onClick`, `onMouseOver`, `onMouseOut`.
Gli eventi intercettabili su una finestra: `onLoad`, `onUnload`, `onBlur`.

4.23 Gestione eventi

Per sfruttare il valore restituito da confirm, prompt o qualsiasi altra funzione JavaScript occorre inserire come valore dell'attributo onClick un programma JavaScript.

```
onClick = "x = prompt('Cognome e Nome:') ;
document.write(x)"
```

4.23.1 Eventi (form)

JavaScript permette di: intercettare eventi che "accadono" nei campi di un modulo e modificare i campi di un modulo.
Un form contiene solitamente campi di testo e bottoni. Quando il bottone viene premuto è possibile invocare una funzione JavaScript.

```
<FORM name="myform">
    <INPUT type="button" name="bottone"
    value="Premi qui"
    onClick = "alert('Mi hai premuto')" >
</FORM>
```

ALTERNATIVA: quando si preme il bottone, far scrivere il risultato di una funzione.

```
<FORM name="myform">
    <INPUT type="button" name="bottone"
    value="Premi qui"
    onClick = "document.write(15)" >
</FORM>
```

Modifica di un campo del messaggio tramite proprietà `onMouseOver` e `onMouseOut`.

```
    <DIV ALIGN="CENTER">
    <FORM NAME="modulo">
        <INPUT TYPE="text" NAME="over" VALUE="Il mouse e sopra
            la scritta1 o la scritta2?" size="50px">
    </FORM>
    <A HREF="" onMouseOver= "window.document.modulo.over.value='scritta1';"
    onmouseout="window.document.modulo.over.value='Il mouse e sopra la
    scritta1 o la scritta2?'">scritta1</A>
    ...
```

Selezionare/deselezionare le checkbox al click su un link/bottone:

```
<FORM NAME="modulo">
 Opzione 1: <INPUT TYPE="checkbox" NAME="box[]" VALUE="v1">
 ...
<input type=button onclick="checkAll(3)"
                    value="Seleziona/deseleziona tutte le opzioni">
```

HTML

```
function checkAll(lungh) {
    for (var i=0; i<lungh; i++) {
        if (document.modulo.elements[i].checked==true)
                {document.modulo.elements[i].checked=false;}
        else if (document.modulo.elements[i].checked==false)
                {document.modulo.elements[i].checked=true;}
    }
  }
```

JavaScript

Impostare dinamicamente le voci di un menu (a), intercettare il click del mouse sul pulsante di invio di un form e di conseguenza mostrare un alert (b).
Menù:

```
<FORM NAME="modulo" onSubmit=" warn(); return false;">
    <SELECT NAME="menu" id="menuid">
        <OPTION ID="fr" VALUE="inter">Inter</OPTION>
        <OPTION ID="pt" VALUE="milan">Milan</OPTION>
        <OPTION ID="st" VALUE="juve">Juve</OPTION>
    </SELECT>
    <INPUT TYPE="Submit" VALUE="OK">
</FORM>
```

return false evita il refresh della pagina che avviene di default con il pulsante submit. Impostare dinamicamente le voci di un menu (a).

```
<SCRIPT language="JavaScript">
    var nazione = prompt("Sei Italiano o Inglese?", "Italiano");
    if (nazione == "inglese") {
        window.document.modulo.menu.options[0].text="Chelsea";
        window.document.modulo.menu.options[1].text="Manchester United";
        window.document.modulo.menu.options[2].text="Manchester City";
        var select = document.getElementById('menuid');
        var opt = document.createElement('option');
        opt.id = "tt";
        opt.value = "liverpool";
        opt.innerHTML = "Liverpool";
        select.appendChild(opt);
}
</SCRIPT>
```

Intercettare il click del mouse sul pulsante di invio di un form e di conseguenza mostrare un alert:

```
<SCRIPT language="JavaScript">
    function warn() {
        if (window.document.modulo.menu.selectedIndex == 0) {
            alert("Very good!");
```

```
    }
}
</SCRIPT>
```

4.23.2 Eventi (onLoad)

È un evento scatenato quando un oggetto viene caricato. Ad esempio pemette di ridirezionare l'utente ad un altro URL.

```
<SCRIPT language="JavaScript">
    function jump(){
        window.location.href="http://www.unimi.it";
    }
</SCRIPT>
</HEAD>
<BODY onLoad = "jump()" >
```

onLoad carica il gestore che viene innescato tramite la funzione `jump()`. Ridirezione all'URL puntata dalla proprietà `href` dell'oggetto `location`.
Ridirezione dopo timout:

```
<HTML>
    <HEAD>
    <SCRIPT language="JavaScript">
        function jump(){
            window.setTimeout("window.location.href='http://www.di.unimi.it/
                ardagna';", 5*1000);
        }
    </SCRIPT>
</HEAD>
<BODY onLoad = "jump()">
</BODY>
</HTML>
```

4.23.3 Eventi (onMouseOver)

L'evento è scatenato al passaggio del mouse su un oggetto. Ad esempio, utilizzato per cambiare lo stile di un oggetto. Un latro utilizzo molto comune è l'image swap (o roll-over), ovvero il cambiamento dell'aspetto di un'immagine.

```
<p ALIGN="CENTER" style="background-color:green" onMouseOver="style='background-
    color:yellow';" onMouseOut="style='background-color:green';">
    Cambia il colore dello sfondo.
</p>
```

onMouseOver/Out cambia il valore src dell'immagine.

4.24 Gestione finestre (open)

L'oggetto `window` permette di agire e gestire le finestre del browser: `window.open(URL, nome, [proprietà])`, apre una nuova finestra (o una nuoca scheda):

- URL: indirizzo della pagina da caricare.
- Nome = identificatore della finestra.
- Proprietà (opzionale): lista delle proprietà della nuova finestra (se omesso, la nuova finestra mantiene le proprietà della corrente).
- Ritorna un riferimento alla finestra aperta (`null` se c'è errore).

Esempio: apertura finestra nella stessa finestra/scheda: `<a href="#" onClick = "window.open('http://www.di.unimi
); return false;">nuova finestra!</a>`
Un altro esempio: `<a href="#" onClick = "window.open('http://www.di.unimi.it', 'pippo', 'scrollbars=yes
status=no,location=no,toolbar=no,menubar=no'); return false;">finestra con proprietà</a>`

- Presenza barre di scorrimento.

- Possibilità di ridimensionare la finestra.
- Larghezza e altezza (in pixel).
- Presenza della barra di stato (status).
- Presenza della barra dei pulsanti (toolbar).
- Presenza della barra del menù.

Se stabilisco delle proprietà. verrà sicurmante aprta una nuova finestra (con quelle proprietà) e NON una scheda.

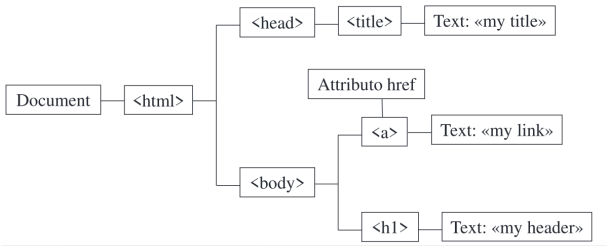
4.25 Funzioni come link

Una funzione JavaScript costituisce un valido link utilizzabile nel tag HTML `<a href=...></a>`. L'effetto del click su tale link è l'esecuzione della funzione e l'apparizione del risultato in una pagina HTML all'interno però della stessa finestra.

4.26 Accesso agli oggetti di una pagina tramite nodi

Ogni oggetto è un nodo (HTML DOM).

- ogni elemento è un nodo;
- ogni testo è un nodo testo;
- ogni attributo è un nodo attributo;
- i commenti sono dei nodi commenti.



4.26.1 Relazione tra nodi

È definita attraverso diverse funzioni:

- parent, child, sibling;
- **root** è la radice del documento (¡html¿);
- ogni nodo ha un solo nodo padre a parte la root;
- ogni nodo può avere 0 o più figli;
- sibling sono nodi con lo **stesso padre**.

Proprietà per la navigazione tra nodi:

- parentNode;
- childNodes[nodenumber];
- fistChild;
- lastChild;
- nextSibling;
- previousSibling.

Accesso al primo nodo figlio: `document.getElementById("intro").childNodes[0]`, oppure `document.getElementById`
Il valore testuale di un elemento è a sua volta un nodo: `document.getElementById("intro").childNodes[0].nodeValue`
Altre proprietà olte a nodeValue: `nodeName`, `nodeType` entrambi in sola lettura.

4.26.2 Aggiunta elementi

Aggiunta di elementi all'albero DOM:

```
<div id="div1">
    <p id="p1">This is a paragraph.</p>
    <p id="p2">This is another paragraph.</p>
</div>
<script>
    var para = document.createElement("p");
    var node = document.createTextNode("This is new.");
    para.appendChild(node);
    var element = document.getElementById("div1");
    element.appendChild(para);
</script>
```

Il metodo `appendChild()` può essere sostituito da `insertBefore()`.

4.26.3 Rimozione degli elementi

Rimozione di elementi dall'albero DOM:

```
<div id="div1">
    <p id="p1">This is a paragraph.</p>
    <p id="p2">This is another paragraph.</p>
</div>
<script>
    var parent = document.getElementById("div1");
    var child = document.getElementById("p1");
    parent.removeChild(child);
</script>
```

4.26.4 Sostituzione eleemtni

Sostituzione di elementi nell'albero DOM:

```
<div id="div1">
    <p id="p1">This is a paragraph.</p>
    <p id="p2">This is another paragraph.</p>
</div>
<script>
    var para = document.createElement("p");
    var node = document.createTextNode("This is new.");
    para.appendChild(node);
    var parent = document.getElementById("div1");
    var child = document.getElementById("p1");
    parent.replaceChild(para,child);
</script>
```

4.26.5 Collezione di nodi

`Node list` è una collezione di nodi: simile agli array di oggetti `images[]` e `forms[]` visti prima. Ad esempio il metodo `getElementsByTagName()` ritorna una lista di nodi. La lista di nodi è una collection di nodi simile ad array. La proprietà `lenght` ritorna la lunghrzza della lista di nodi.

4.26.6 Metodo getElementsByClassName

Il metodo `getElementsByClassName(classname)` ritorna una lista NodeList di tutti gli elemetni con una specifica classe. Gli eleemtni in NodeList sono acceduti tramite indici.

```
<div class="example">First div element with class="example".</div>
<div class="example">Second div element with class="example".</div>
var x=window.document.getElementByClassName("example")
document.write(x[0].innerHTML)
```

4.27 Funzioni

4.27.1 Introduzione

Object-based language (ma non object-oriented): usa l'idea di incapsulare stato e operazioni all'interno di oggetti, non applicando però i concetti di ereditarietà e sottotipi. JavaScript a volte riperito come prototype-based language: non esistono classi, ma gli oggetti ereditano codice e dati da altri oggetti template. Vengono clonati oggetti che servono come "prototipi".

4.27.2 Definizioen di funzioni

Le funzioni sono definite tramite keyword `function` e sempre racchiuso in un blocco. Posso essere considerate sia **procedure**, sia **funzioni** in senso proprio. I parametri formali sono senza dichiarazione di tipo.

4.27.3 Chiamata a funzione

Le funzioni sono chiamate come in un linguaggio di programmazione tradizionale, fornendo la lista dei parametri attuali. Se i tipi non hanno senso per le operazioni fornite, l'inerprete JavaSCript ritorna un errore a runtime. Non viene mostrato il risultato.

4.27.4 Parametri di funzione

Passaggio di parametro per valore: nel caso di oggetti, si copiano i riferimenti. A differenza di C e Java, è lecito definire una funzione dentro un'altra funzione. Se i parametri attuali sono più di quelli necessari, non c'è nessun errore, semplicemente vengono ignorati. Se i parametri attuali sono meno di quelli necessari, quelli mancanti sono inizializazti a `undefined`.

4.28 Variabili: tipi di dichiarazione

La dichiarazione delle variabili è:

- implicita o esplicita per le variabili globali;
- necessariamente **esplicita**, per variabili locali.

4.28.1 Variabili: dichiarazione esplicita

FUori da funzioni, la parole chiave `var` è ininfluente: la variabile definita è **globale**. All'interno di funzioni, la parole chiave `var` ha un significato preciso: indica che la nuova variabile è locale, ossia ha come scope la funzione. All'interno di funzioni, una dichiarazione senza la parola chiave `var` introcuce una variabile globale.

```
x=6 // globale
function test(){
  x = 18 // globale
}
test()
// qui x vale 18
```

```
var x=6 // globale
function test(){
  var x = 18 // locale
}
test()
// qui x vale 6
```

4.28.2 Variabili: environment di riferimento

Quando ci si riferisce a una variabile, prima si cerca **localmente** e se non è definita si accede a quella **globale**.
Il parser prima analizza tutto il codice e crea tutte le variabili e strututre lasciandole `undefined`. Poi esegue una riga alla volta.

4.29 Funzioni e chiusure

La natura interpretata di JavaScript e l'esistenza di un ambiente globale pongono una domanda: quando una funzione usa un simbolo non definito al suo interno, quale definizione vale per esso? La definizione che esso ha nell'ambiente in cui la funzione è definita, oppure la definizione che esso ha nell'ambiente in cui la funzione è chiamata?
Si consideri il seguente programma:

```
var x = 20;
function provaEnv(z) { return z + x; }
alert(provaEnv(18)) // visualizza certamente 38
function testEnv() {
    var x = -1;
    alert(provaEnv(18)); // COSA visualizza ???
}
```

Nella funzione `testEnv` si ridefinisce il simbolo x, poi si invoca la funzione `provaEnv`, che usa il simbolo x, ma quale x? Nell'ambiente in cui provaEnv è definita, il simbolo x aveva un altro significato rispetto a quello che ha ora! Se vale l'ambiente esistente all'atto dell'uso di provaEnv, si parla di chiusura **dinamica**; se prevale l'ambiente di definizione di provaEnv, si parla di chiusura **lessicale**. JavaScript adotta la chiusura lessicale.

4.29.1 Funzioni come dati

Variabili possono riferirsi a funzioni: la funzione non ha un nome: `var f = function (z) return z*z;` . La funzione viene invocata tramite il nome della variabile:

```
var result = f(4);
g = f produce aliasing
```

È possibile passare funzioni come prametro ad altre funzioni.
Ciò cosegue varie conseguenze:

- per utilizzare una funzione come dato occorre avere effettivamente un oggetto funzione;
- non si può sfruttare questa caratteristica per far eseguire una funzione di cui sia noto solo il nome;
- o di cui sia noto solo il codice;
- il problema è risolvibile: si costruisce esplicitamente un "oggetto funzione", oppure si accede alla funzione tramite le proprietà dell'oggetto globale (si usa il primo).

4.30 Oggetti

Un oggetto JavaScript è una collezione di dati dotata di nome:

- ogni dato interpretabile come una proprietà:
- per accedere alle proprietà si usa la "dot notation": `nomeOggetto.nomeProprietà`;
- tutte le proprietà sono accessibili.

Un oggetto JavaScript è costruito tramite **costruttore**: sstabilisce la struttura dell'oggetto e quindi le sue proprietà. I costruttori sono invocati mediante l'operatore `new`. In JavaScript non esistono classi e sottoclassi. Il nome del costruttore è a scelta dell'utente. La struttura di un oggetto non è stabilita dalla classe.

4.30.1 Definizione

La struttura di oggetto JavaScript viene definita dal costruttore usato per crearlo. È all'interno del costruttore che si specificano le proprietà (iniziali) dell'oggetto, elencandole con la dot notation e la keyword `this`.
Identificatore **globale** (function expression):

```
Point=function(i,j){
    this.x=i;
    this.y=j;
}
```

Identificatore **locale** (function declaration):

```
function Point(i,j){
    this.x=i;
    this.y=j;
}
```

La keyword `this` è necessaria, altrimenti ci si riferirebbe all'environment locale della funzione costruttore.

4.30.2 Costruzione

Per costruire oggetti si applica l'operatore `new` a una funzione costruttore:

```
p1 = new Point(3,4);
```

A partire da JavaScript 1.2, si possono creare oggetti anche elencando direttamente le proprietà con i rispettivi valori: sequenza di coppie nome:valore separate da virgole e racchiusa fra parentesi graffe: `p3= x:10, y:7`.

4.30.3 Accesso alle proprietà

Le proprietà di un oggetto sono pubbliche e accessibili: esistono anche proprietà "di sistema" e come tali non visibili, nè enumerabili con gli appositi costrutti. Accesso attraverso la dot notation: `p1.x=10`.

4.30.4 Aggiunta e rimozione di proprietà

Le proprietà specificate nel costruttore sono le proprietà **iniziali**. È possibile aggiungere dinamicamente nuove proprietà semplicemente nominandole e usandole. `p1.z=-3`. È possibile rimuovere dinamicamente proprietà, mediante l'operatore `delete`: `delete p1.x`.

4.30.5 Metodi per (singoli) oggetti

Definire metodi è semplicemente un caso particolare dell'aggiunta di proprietà. Non esistendo il concetto di classe, un metodo viene definito per uno specifico oggetto e non per tutti gli oggetti della stessa "classe"-

```
p1.getX = function() { return this.x; }
```

4.30.6 Metodi per una "classe" di oggetti

In assenza del concetto di classe, assicurare che oggetti "dello stesso tipo" abbiano lo stesso funzionamento richiede un'opportuna metodologia. Un possibile approccio consiste nel deifnire tali metodi dentro al costruttore:

```
Point = function(i,j) {
    this.x = i; this.y = j;
    this.getX = function(){ return this.x }
    this.getY = function(){ return this.y }
}
```

4.30.7 Metodi: invocazione

L'operatore di chiamata () è quello che effettivamente invoca il metodo: `document.write(p1.getX()+"<br/>")`. ATTENZIONE: se si invoca un metodo inesistente si ha un errore a run-time. NB: se l'interprete JavaSCript incontra un errore a run-time, non esegue le istruzioni successive e spesso non visualizza alcun messaggio di errore!

4.31 Oggetti Function

Gli oggetti Function permettono di definire **funzioni**. Ogni funzione JavaScript è un oggetto:

- definizione **implicita** tramite il costrutto `function`;
- definizione **esplicita** tramite il costruttore `Function`.

4.31.1 Definizione implicita

Nella definizione implicita gli argomenti sono i parametri formali della funzione. Il corpo della funzione è racchiuso tra parentesi graffe:

```
funzione=function(x){ return f(x)}
```

Viene costruito dentro il programma JavaScript: valutato una sola volta, è efficiente, ma non flessibile.

4.31.2 Definizione esplicita

Nella definizione esplicita gli argometni sono tutte le stringhe e rappresentano i parametri della funzione definita. Solo l'ultimo argomento ha un comportamento diverso e rappresenta il corpo della funzione.

```
funzione= new Function("x","return f(x)")
```

Viene costruito a partire da stringhe. Valutato ogni volta, è poco efficiente, ma molto flessibile.

4.32 Funzioni come dati: Esempio

Funzione on demand:

- Inserire la funzione da calcolare:

```
    var funzione = prompt("Scrivere f(x): ")
```

- Inserire il/i parametri da usare:

```
    var x = prompt("Calcolare per x = ? ")
```

- Calcolare la funzione (invocazione riflessiva):

```
    var f = new Function("x", "return " + funzione)
```

- Mostrare il risultato:

```
    confirm("Risultato: " + f(x))
```

4.32.1 Funzioni come dati: problema

I valori immessi da linea di comando (prompt) sono stringhe. Contromisure:

- l'utente specifica il tipo del dato attraverso una conversione esplicita;
- il programma implementa la conversione esplicita dopo il prompt:

```
    var x = parseInt(prompt("Calcolare per x = ? "))
    typeof(x) = number
```

4.33 Oggetti Function: proprietà

Proprietà **statiche** (esistono anche mentre non esegue):

- `length` - numero di parametri formali (attesi).

Proprietà **dinamiche** (mentre la funzione è in esecuzione):

- `arguments` - array contente i parametri attuali;
- `arguments.length` - numero dei parametri attuali;
- `arguments.callee` - la funzione stessa in esecuzione;
- `caller` - il chiamante (`null` se invocata da top level);
- `constructor` - riferimento all'oggetto costruttore;
- `prototype` - riferimento all'oggetto prototipo.

4.34 Oggetti Function: metodi

Metodi invocabili sun una funzione:

- `toString` - chiamata automaticamente quando la funzione deve essere rappresentata come testo: ritorna una rappresentazione a stringa dell'oggetto funzione;
- `valuesOf` - ritorna la funzione stessa come oggetto;
- `call` e `apply` - funzioni applicate all'oggetto passato com primo argomento (identifica il contesto this), fornendo a tale oggetto i restanti parametri:

```
    funz.apply(ogg, arrayDiParametri )
```

```
    funz.call(ogg, arg1, arg2, ... )
```

4.35 Funzioni come dati - Conseguenze

Per utilizzare una funzione come dato occorre avere effettivamente un oggetto funzione. Non si può sfruttare questa caratteristica per far eseguire una funzione di cui sia noto solo il nome (letto da tastiera), o di cui sia noto solo il codice. Il problema è risolvibile accedendo alla funzione tramite le proprietà dell'oggetto globale.

4.36 Oggetto globale

PROBLEMA: come può JavaScript distringuere fra metodi di oggetti e funzioni "globali"? Semplicemente non distingue: le funzioni "globali" non sono altro che metodi di un "oggetto globlae" definito dal sistema. L'oggetto "globale" ha come metodi le funzioni non attribuite a uno specifico oggetto nonchè a quelle globali; come dati, le variabili incluse quelle predefinite; oggetti predefiniti.

4.37 [Costruttori di] Oggetti predefiniti

Oggetti di uso generale:

- Array, Boolean, Function, Number, Object, String.
- L'oggetto Math contiene la libreria matematica: costanti e funzioni di ogni tipo. Non va istanziato ma usato come componente "statico".
- L'ggetto Date definisce i concetti per esprimere date e orari e lavorare su di essi: va istanziato nei modi opportuni.
- L'oggetto REgExp fornisce il supporto per le espressioni regolari.

Oggetti di uso grafico: Anchor, Applet, Area, Button, Checkbox, Document, Event, FileUpload, Form, Frame, Hidden, History, Image, Layer, Link, Location, Navigator, Option, Password, Radio, Reset, Screen, Select, Submit, Text, Textarea, Window

4.38 Date

4.38.1 Costruzione

Costruttore:

- Date(), Date(millisecondi), Date(stringa), Date(anno, mese, giorno [, hh, mm, ss, msec]).
- Date(): ciene creato un oggetto corrispondente alla data e all'ora correnti, come risultano sul sistema in uso.
- Date(millisecondi): i millesecondi sono calcolati dalle ore 00:00:00 del 1° gennaio 1970 usando il giorno standard UTC di 86.400.000 ms.
- Date(string): string è nel formato riconosciuto da Date.parse.
- Date(anno, mese, giorno, ...): anno, mese e giorno devono essere forniti, gli altri sono opzionali.

4.38.2 Metodi

Metodi:

- `getDay`: restituisce il giorno della settimana, da 0 (dom) a 6 (sab)
- `getDate`: restituisce il numero del giorno, da 1 a 31
- `getMonth`: restituisce il mese, da 0 (gennaio) a 11 (dicembre)
- `getFullYear`: restituisce l'anno (su quattro cifre)
- `getHours`: restituisce l'ora, da 0 a 23
- `getMinutes`: restituisce i minuti, da 0 a 59
- `getSeconds`: restituisce i secondi, da 0 a 59

4.39 Oggetto globale

L'oggetto "globale" è **UNICO** e viene sempre creato dall'interprete prima di eseguire alcunchè. Però non esiste un identificatore "global": in ogni situazione c'è un dato oggetto usato come globale.
In un browser Web, l'ggetto globale solitamente coincide con l'oggetto `window`, ma non sempre è così: a lato server, per esempio, sarà probabilmente l'oggetto response a svolgere quel ruolo. Quindi, in un browser, per scoprire tutte le proprietà dell'oggetto globale, basta invocare `show(window)`.
Oltre all'apporccio basato sul costruttore `Function`, si può sfruttare l'oggetto globale per ottenere un riferimento all'oggetto funzione corrispondente a un dato nome di funzione purchè la funzione richiesta sia già definita nel sistema. Se p è un riferimento a un oggetto, e s è il nome di una sua proprietà x, la notazione "array-like" $p[s]$ fornisce un riferimento all'oggetto (proprietà) x. Ad esempio, la notazione var math = Math; var nome = math["sin"]

Pone nella variabile nome un riferimento all'oggetto funzione Math.sin (Nota: l'assegnamento math = Math è necessario perché la notazione array-like è ammessa solo su variabili, e Math non lo è)
A seguito di ciò, definita la funzione function calc(f,x) return f(x) È ora possibile effettuare l'invocazione calc(nome, .8) che dà 0.7173560908995228 Il nome "sin" viene trasformato in un riferimento all'oggetto Math.sin, utilizzabile per la chiamata

4.40 API

Per eseguire una fetch dobbiamo:

```
fetch('link')
    .then(response => response.json())
    .then(data => console.log(data))
```

dove response è un oggetto che rappresenta la risposta alla richiesta HTTP, e data è un oggetto che rappresenta i dati restituiti dal server. Traduce da json ad oggetto js.

5 Node.js

Node.js è un ambiente JavaScript multipiattaforma lato server. JavaScript è il linguaggio utilizzato come "**collante**" per i moduli C++. È basato sul motore JavaScript V8 di Google.
Node.js utilizza un modello di I/O **non bloccante** e guidato dagli **eventi**. Node.js è come un browser web con un diverso insieme di moduli C++. Fornisce moduli per interagire con le risorse del sistema, come i processi, il file system, la rete, ecc. . . .

5.1 Programmi

I programmi Node.js possono essere creati utilizzando file JavaScript che devono essere eseguiti tramite Node.js. I file contengono codice JavaSCript e hanno estension "js". Per eseguire un programma, è necessario eseguire Node.js e passare il nome del file (con o senza l'estensione js) come primo parametro.

5.2 Gestire pacchetti con NPM

NPM è l'acronimo di **Node.js Package Manager**. Esso fornisce:

- un repository pubblico di pacchetti;
- una specifica per la creazione di pacchetti;
- uno strumento a riga di comando per lavorare con i pacchetti.

La società npm, Inc. sviluppa e mantiene NPM.

5.3 Pacchetti locali vs globali

I pacchetti possono essere installati **localmente** o **globalmente**. I pacchetti **locali** sono memorizzati localmente nella cartella `node_modules`. In genere i pacchetti locali sono librerie di codice utilizzate dal progetto. I pacchetti **globali** sono memorizzati globalmente sul sistema.
I pacchetti locali sono disponibili solo all'interno del progetto specifico, mentre quelli globali sono disponibili in tutto il sistema.

5.3.1 Installare e disinstallare pacchetti

Il programma npm viene utilizzato per **gestire** i pacchetti. Il primo argomento del programma npm è il comando da eseguire. I pacchetti possono essere installati ocn il comando `install` e disinstallati con il comando `unistall`. Esistono molti altri comandi disponibili per NPM.

```
$ npm install --global express

$ npm install express

$ npm uninstall --global express

$ npm uninstall express
```

- Il flag **--global** installa e disinstalla il pacchetto a livello globale; senza il flag global, i pacchetti vengono installati e disinstallati localmente.

5.4 Creare un semplice Server Web

Uno dei moduli principali per Node.js è il modulo **HTTP** che fornisce un server Web e un client Web. Il **server Web** è molto flessibile, ma richiede molto codice boilerplate per costruire anche le applicazioni più semplici. Comunemente, altri pacchetti come Express vengono utilizzati per configurare il server Web. I server Web sono applicazioni ad alta intensità I/O che li rendono adatti per Node.js, il quale è ottimo grazie alla sua facile gestione dei dati JSON.

5.5 Leggere i file dal disco

Node.js consente l'**accesso completo** al sistema a differenza di un browser Web che consente solo l'accesso in modalità sandbox. L'accesso alle disorse del file system può avvenire in modo **sincrono** o **asincrono**. L'accesso sincorno può essere utilizzato per il caricamento iniziale del programma, ma solo l'accesso asincrono deve essere utilizzato durante il funzionamento del programma. Sia il testo che i dati binari possono essere letti e scritti.

5.6 Creare un pacchetto

Tutti i progetti (che sono anche pacchetti) devono essere configurati per funzionare con NPM. Il comando `npm init` viene utilizzato per configurare un progetto. Verrà oista una serie di domande, tutte con risposte predefinite, utilizzate per creare e inizializzare un file `package.json`.
Il file `package.json` contiene metadati sul progetto, nonchè un elenco di **dipendenze** dell'applicazione e dello sviluppo. Quando i pacchetti NPM sono installati, NPM li salva nel file `package.json`.

5.6.1 Salvare le dipendenza di un pacchetto

La semplice installazione dei pacchetti non salva la dipendenza nel file `package.json`. Oltre all'installazione, è necessario specificare flag aggiuntivi:

- `--save` o `-S` salverà il pacchetto come dipendenza nell'**applicazione**;
- `--save-dev` o `-D` salverà il pacchetto come dipendenza per lo **sviluppo**.

Le dipendenze dell'applicazione vengono utilizzate dal programma Node.js durante l'esecuzione. Le dipendenze per lo sviluppo vengono utilizzate per sviluppare il programma Node.js.

Terminal Commands

```
$ npm init -y

$ npm install -S express

$ npm install -D grunt

$ more package.json
```

Package.json

```
{
  "name": "demo",
  "version": "1.0.0",
  "description": "",
  "main": "Gruntfile.js",
  "dependencies": {
    "express": "^4.13.4"
  },
  "devDependencies": {
    "grunt": "^1.0.1"
  }
}
```

I comandi del terminale produrranno un file `package.json`. IL file è un file JSON e può essere modificato manualmente.
`Name` è il nome del pacchetto. La versione segue lo schema SEMVER. Viene monitorata anche la versione di ogni dipendenza. Main è il file principale importanto quando si richiede il modulo.

5.7 Usare NPM per eseguire un pacchetto

Oltre ad essere gestiti NPM, i pacchetti possono essere eseguiti con NPM. Per configurare l'esecuzione dei pacchetti Node, viene configurata l'opzione `scripts` del file `package.json`. Per eseguire un programma Node.js, lo script di avvio è configurato.